VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

SCHOOL OF COMPUTER SCIENCE ENGINEERING AND INFORMATION SYSTEMS (SCORE)

FALL SEMESTER 2025-26

REVIEW-03

SWE1901 –TECHNICAL ANSWERS FOR REAL WORLD PROBLEMS (TARP)

TITLE: IMAGE BASED FOUNDATION SHADE RECOMMENDATION SYSTEM USING ML

Submitted by:

- **PAVITHRA S – 22MIS0180**
- **PREETHI P – 22MIS0348SS**
- **YOGASHRI S- 22MIS0572**

Under the guidance of

Dr. Ramkumar T

- **OBJECTIVES OF THE PROJECT:**

  To engineer a comprehensive, full-stack web application that solves the challenge of remote cosmetic matching by integrating a robust hybrid computer vision and machine learning pipeline for precise skin tone analysis, while ensuring a secure, personalized, and engaging user experience through modern accessibility features and cloud-based persistence.

- **PROBLEM STATEMENT:**

  The shift towards e-commerce has made purchasing complexion products, such as foundation, notoriously difficult. Unlike physical stores where testers are available, online shoppers must rely on inaccurate screen representations of colors. Standard digital images often fail to capture true skin tone due to three critical factors: variable ambient lighting, poor camera quality, and user inability to accurately self-identify undertones. This leads to high rates of consumer dissatisfaction, wasted money on mismatched products, and significant reverse-logistics challenges for retailers. Existing online tools are often overly simplistic quizzes that lack personalization or proprietary brand tools that lack transparency. There is a need for a universal, technically robust system that can "see" past bad lighting to find a user's true shade.

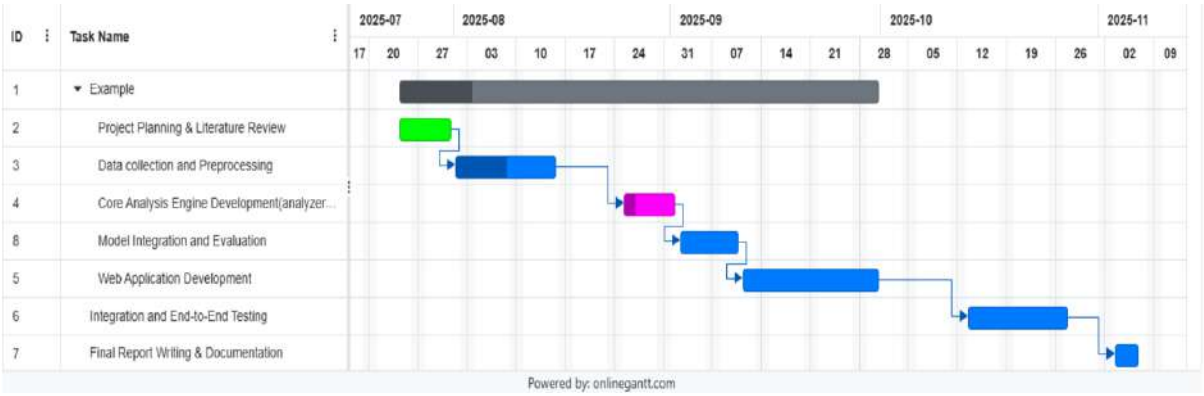
- **MOTIVATION IN TERMS OF TECHNICAL SOCIETAL/ENVIRONMENTAL/DEMOGRAPHICAL PERSPECTIVE:**

  Technical Motivation: The project tackles the classic, complex computer vision challenge of "color constancy"—determining the true color of an object (skin) regardless of the light source illuminating it. Motivated by the limitations of simple average-color extraction, this project seeks to apply advanced techniques like K-Means clustering and deep-learning-based landmark detection (MediaPipe) to turn noisy, unstructured user photos into precise, structured color data.

  Societal & Demographical Motivation: Finding the right foundation shade is deeply personal and often frustrating, particularly for individuals with underrepresented skin tones who historically face limited options. By using a data-driven approach rather than subjective guesswork, this tool aims to democratize access to professional-level shade matching. It caters specifically to a digitally-native demographic that demands personalization, speed, and accuracy in their online shopping experience.

Environmental Motivation: The beauty industry generates significant waste through product returns. A major portion of online cosmetics returns are due to incorrect shade selection. Often, these returned products cannot be resold due to hygiene reasons and end up in landfills. By increasing the "first-time-right" purchase rate through accurate AI recommendations, this project aims to contribute to reducing the carbon footprint associated with manufacturing, shipping, and disposing of mismatched products.

- **TIMELINE OF ACTIVITIES ALONG WITH OUTCOMES - GANTT CHART:**



- **LITERATURE REVIEW – SUMMARY OF LITERATURE STUDIED / GAP IDENTIFIED:**

Existing remote cosmetic matching solutions predominantly rely on two approaches: subjective questionnaire-based quizzes or simplistic average-color extraction from user-uploaded images. Literature in computer vision indicates that simple RGB averaging is highly susceptible to ambient lighting noise (the "color constancy" problem) and often incorrectly includes non-skin pixels (hair, background, shadows), leading to inaccurate results. Furthermore, many commercial solutions are proprietary "black boxes" restricted to single brands. **Gap Identified:** There is a lack of transparent, vendor-neutral systems that robustly combine advanced computer vision techniques (to neutralize lighting and precisely isolate skin texture) with a hybrid decision-making model that cross-references machine learning predictions with mathematical color theory for higher accuracy.

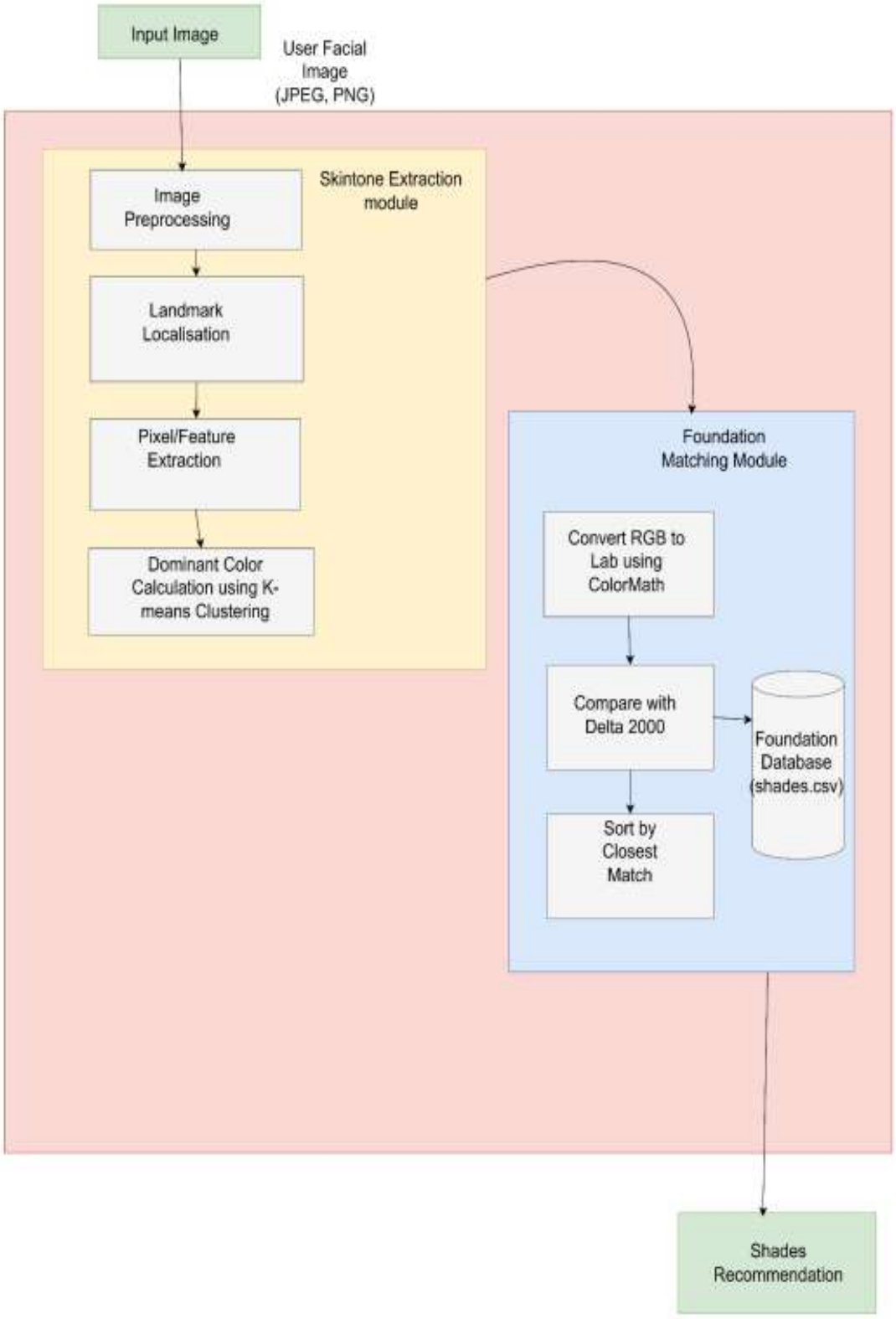
S. No	Title	Author(s)	Year & Publication	Abstract	Benefits	Limitations
1	A Color Image Analysis Tool to Help Users Choose Makeup	T. Researcher et al.	2024, arXiv / Conference	Describes a pipeline that uses device color-calibration and supervised learning to predict how a foundation shade will appear on a user's face from a no-makeup selfie and product swatch images.	Improves shade-match accuracy by compensating for camera/lighting differences; practical calibration step.	Requires user to include calibration card or follow strict capture protocol — less convenient for casual users.
2	Scalable & Realistic Virtual Try-On for Foundation	VTO Team	2025, arXiv / Preprint	Presents a deep rendering network that simulates realistic foundation application on facial images, enabling visual virtual try-on (VTO) combined with shade recommendation.	Visual confirmation reduces mis-selection and return rates; enhances user trust.	High compute and data needs; color fidelity across devices remains challenging.
3	Industry AI Shade-Matching Service (case study)	AmorePacific / Industry report	2024–25, News/Industry Docs	Case study of an industry deployment integrating facial scans, shade catalogues and on-site mixing to deliver custom foundation matches to customers.	Demonstrates commercial viability and end-to-end fulfillment; rich real-world data.	Proprietary; limited public method details and evaluation metrics.
4	Improving Recommendations by Assessing Face	Corporate R&D (Amazon Science style)	2023, Industry blog/paper	Introduces an illumination-quality classifier that filters or corrects poorly lit selfies before	Addresses a major practical confounder (bad lighting), improving	Focuses on illumination only — additional color calibration still

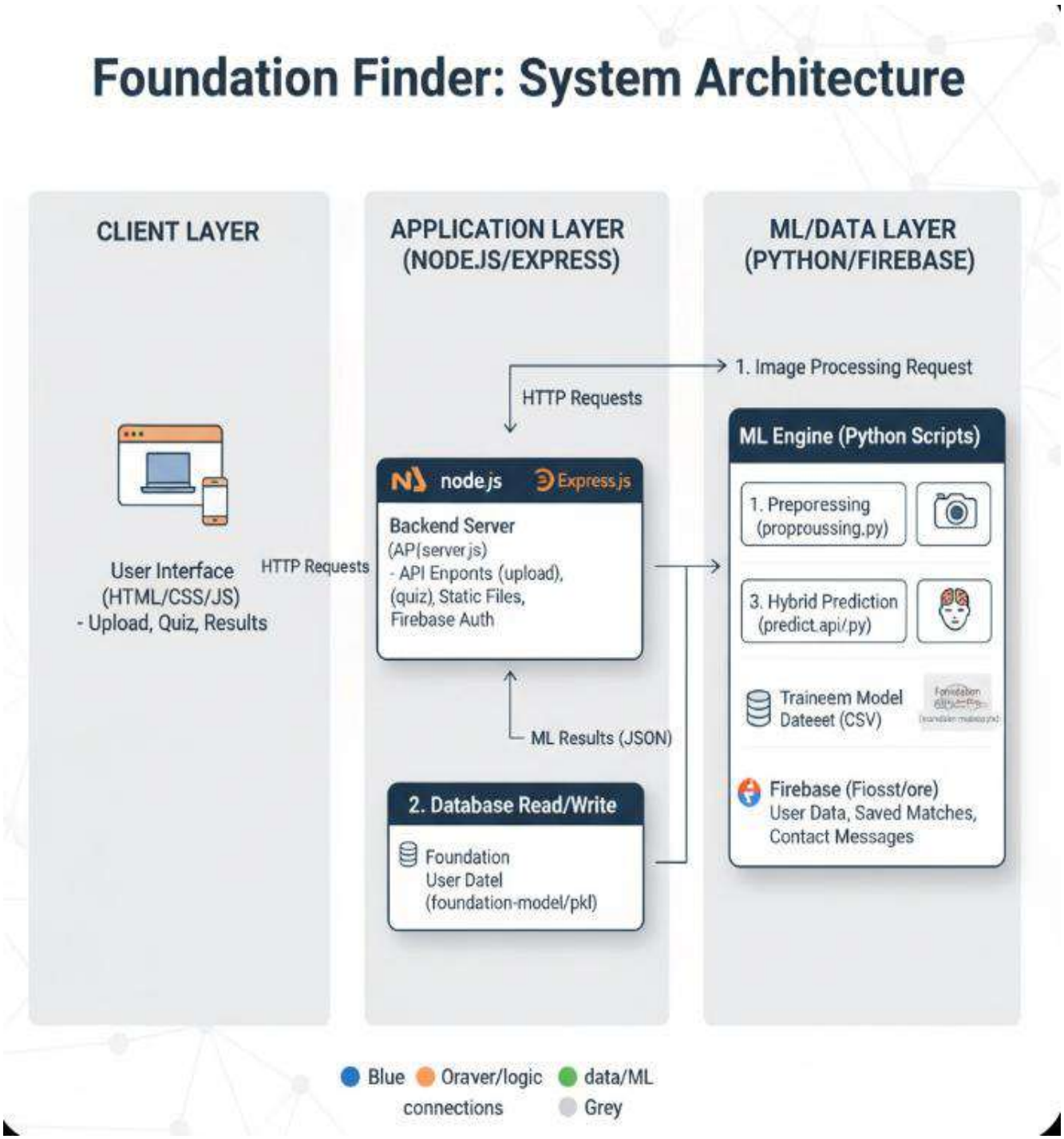
	Illumination Quality			feeding them to downstream recommenders.	downstream accuracy.	needed for best results.
5	Automated Skin Tone Assignment using ITA and CIELAB	M. SkinTone et al.	2022, Computer Vision Workshop	Compares automated ITA/CIELAB-based skin tone measures against human raters on multiple datasets; shows strong agreement after color correction.	Provides a colorimetric basis for mapping skin to shade spaces; reproducible numeric features.	Accuracy depends heavily on color calibration; real-world selfies vary widely.
6	Skin Tone Estimation under Diverse Lighting	Large Volunteer Study	2023, Open Access (journal)	Trains CNNs on a broad, diverse image collection to estimate skin tone robustly across illumination and camera variability.	Demonstrates lighting-robust models given diverse training data; emphasizes dataset variety.	Mapping estimated skin tone to brand shade palettes is an extra nontrivial step; datasets must be broad.
7	Makeup Shade Datasets & Commercial APIs	PerfectCorp / Public Datasets	2021–2024, Industry & Kaggle	Describes available commercial APIs and public shade inventories (brand shade lists with color codes) used by researchers and developers.	Readily available shade catalogs accelerate prototype development and product matching.	Data quality and labeling consistency vary across brands; many APIs are proprietary/paid.

- **OVERCOMING OF LIMITATIONS IF ANY THROUGH THE PROPOSED WORK:**

Identified Limitation in Existing Systems	Proposed Technical Solution in this Work
Variable Lighting Conditions: Standard user photos often suffer from color casts (e.g., overly yellow indoor light or blue outdoor shade), skewing skin tone analysis.	**Advanced Preprocessing Pipeline:** Implemented Gray-World White Balancing to neutralize color casts and a specular highlight removal algorithm to eliminate shiny spots before analysis.
Noise from Non-Skin Pixels: Simple tools average the entire image, incorrectly including hair, background, eyes, and clothing in the skin tone calculation.	**Intelligent Feature Extraction:** Utilized Google MediaPipe Face Mesh to identify 468 precise facial landmarks, enabling exclusive extraction of pixels only from key skin regions (cheeks and forehead).
Single-Method Inaccuracy: Relying solely on one mathematical rule (like nearest neighbor) can be easily thrown off by slight camera sensor variations or shadows.	**Hybrid Recommendation Engine:** Developed a dual-system approach that uses a trained Machine Learning classifier for primary intuition and cross-verifies it against strict mathematical Euclidean distance for maximum accuracy.

- **HIGH-LEVEL CONCEPTUAL FRAMEWORK OF THE PROPOSED WORK:**





1. **Skintone Extraction Module (Analysis):**

   This module's purpose is to analyze the user's input image to derive a single, dominant skin tone hex code, along with a score to validate its reliability. The process flows through five distinct steps:

   1. **Input & Preprocessing:** The system accepts a user's facial image (JPEG/PNG). It is immediately preprocessed to standardize its size, perform "Gray-World" white balancing to correct for environmental

lighting casts, and apply a mask to remove specular highlights (shiny spots) that are not representative of true skin color.

2. **Landmark Localisation:** The clean, preprocessed image is fed into the MediaPipe Face Mesh model, which localizes 478 key facial landmarks.

3. **Pixel/Feature Extraction:** Using these landmarks, the system intelligently isolates specific Regions of Interest (ROIs)—namely the forehead and cheeks, which are stable indicators of skin tone. This step extracts the raw skin pixels from these regions, ignoring hair, eyes, and lips.

4. **Dominant Color Calculation:** The entire collection of extracted skin pixels is fed into a **K-Means Clustering** algorithm. This algorithm groups the pixels into three dominant color clusters, selecting the center of the largest cluster as the single most representative skin tone.

5. **Confidence Score Generation:** In parallel, the system calculates the dominant color for *each ROI separately* (forehead, left cheek, right cheek). It then measures the **Euclidean distance** in RGB space between these regional colors. A large distance indicates uneven lighting (shadows), resulting in a low confidence score, while similar colors (even lighting) yield a high score.

2 Foundation Matching Module (Recommendation)

The hex code and confidence score from the first module are passed to this second module, which compares the user's skin tone against a real-world product database.

1. **Color Space Conversion:** The extracted RGB hex code is converted into the **CIELAB (Lab)** color space. This is a critical step, as CIELAB is designed to mimic human color perception, making comparisons far more accurate than simple RGB.

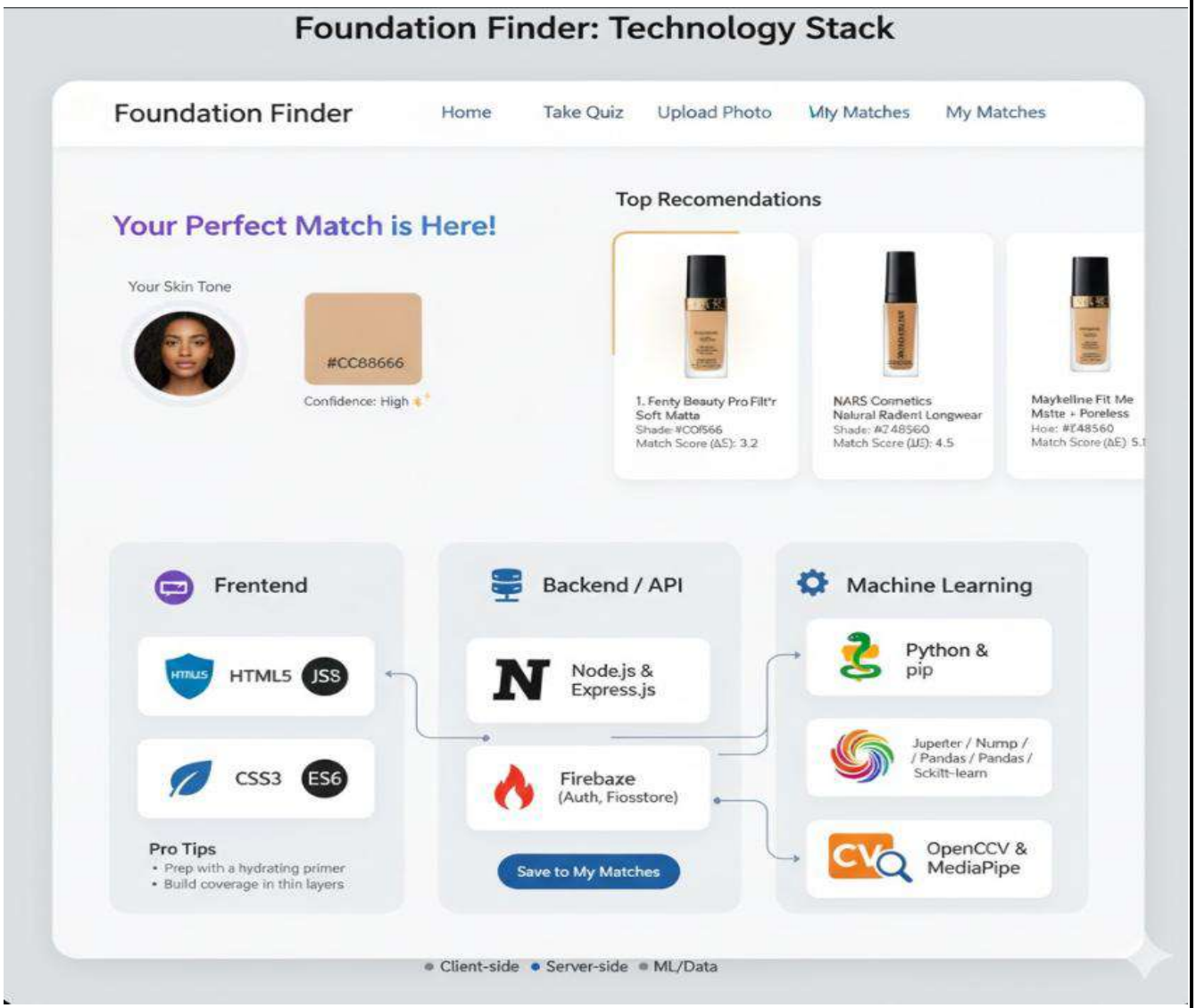
2. **Database Comparison:** The system loads the **Foundation Database** (from dataset_preprocessed.csv).

3. **Perceptual Matching (Delta E):** It iterates through every foundation, converts its hex code to the CIELAB space, and calculates the perceptual difference between the user's color and the product's color using the **deltaE_cie76** formula (from scikit-image). This formula returns a single number (Delta E) representing the "distance" between the two colors as a human would see them.

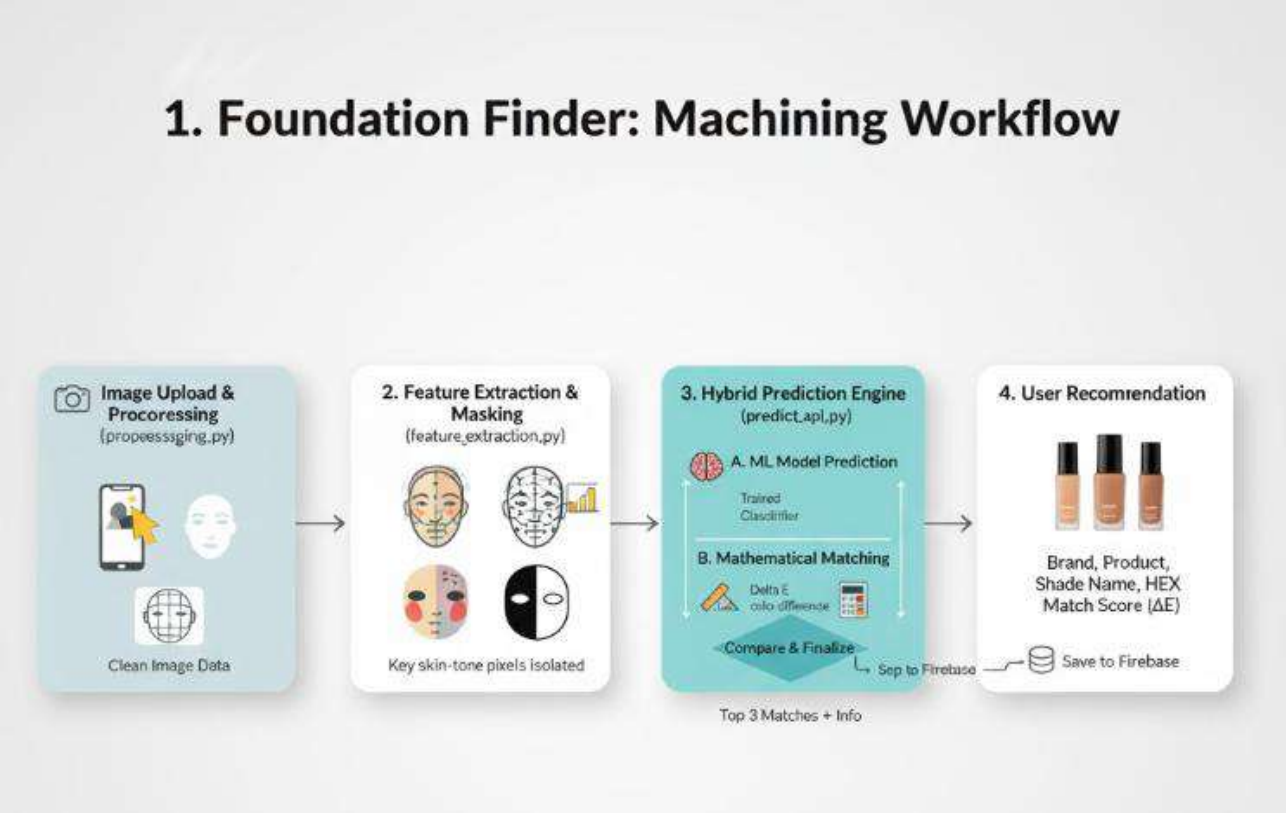
4. **Sort & Recommend:** All products are sorted by their Delta E score, from lowest (best match) to highest. The framework then presents the Top 3 products as the final

5. **Shades Recommendation**s

   A key feature of this entire framework is its **built-in visual validation pipeline**, which displays the output of each major step (Preprocessing, Landmarks, ROIs, and K-Means Palette), allowing for transparent verification of the analysis process



- **BRIEF DESCRIPTION OF THE MODULES INVOLVED IN THE CONCEPTUAL DESIGN:**



1. Image Preprocessing Module

This module is responsible for normalizing raw user inputs to ensure consistent analysis regardless of the source camera or lighting conditions.

- **Purpose:** To standardize the input image and mitigate environmental noise.
- **Key Functions:**
  - **Standardization:** Resizes images to a uniform scale while preserving aspect ratio for consistent processing speed.
  - **Color Correction:** Applies the **Gray-World assumption** algorithm to neutralize color casts caused by ambient lighting (e.g., overly yellow indoor lights).
  - **Highlight Removal:** Detects and masks specular highlights (shiny spots due to oil or flash) that represent pure light reflection rather than true skin pigmentation.

2. Feature Extraction & Analysis Module

This intelligent module uses computer vision to locate facial features and extract a robust skin tone sample.

- **Purpose:** To isolate true skin pixels and determine the user's dominant undertone with a reliability metric.
- **Key Functions:**
    - **Landmark Detection:** Utilizes **MediaPipe Face Mesh** to map 478 3D facial landmarks, enabling precise localization of facial geometry.
    - **ROI Targeting:** dynamically generates masks for stable skin regions (specifically the forehead and cheeks) using predefined landmark indices, avoiding transient features like eyes, lips, and hair.
    - **Dominant Color Clustering:** Applies **K-Means clustering** to the extracted skin pixels to mathematically determine the most representative skin tone, ignoring minor outliers.
    - **Confidence Scoring:** Calculates the variance in skin tone across different ROIs to generate a confidence percentage, providing feedback on lighting uniformity.

3. Shade Matching & Recommendation Module

The final module acts as the recommendation engine, using color science principles to bridge the gap between digital analysis and physical products.

- **Purpose:** To accurately map the extracted digital skin tone to commercially available foundation shades
- **Key Functions:**
    - **Color Space Conversion:** Transforms both user extracted RGB values and database hex values into the **CIELAB (Lab)** color space. CIELAB is used because it is designed to approximate human vision, unlike RGB.
    - **Perceptual Difference Calculation:** Utilizes the **Delta E (CIE 2000)** formula to calculate the mathematical "distance" between the user's skin tone and every product in the database.
    - **Ranking Engine:** Sorts all potential matches by their Delta E score and returns the top 3 products with the lowest perceptual difference as the final recommendations.

- **DATA PROCESSING STANDARDS IMPLEMENTED:**

 Color Space Standards: Conversion between **RGB** (for display), **BGR** (for OpenCV processing), **HSV** (for skin-tone filtering), and **LAB** (experimentally used for deeper color difference calculation, Delta E).
 Data Interchange: All communication between the Python ML engine and Node.js server uses standard **JSON** (JavaScript Object Notation) for structured, language-agnostic data transfer.

 Image Standards: Accepts standard formats (JPG, PNG, WEBP) and internally standardizes them to a fixed long-edge resolution (e.g., 800px) to ensure consistent processing speed regardless of input camera quality.
 Security Standards: Utilizes **Firebase Authentication** protocols for handling user credentials, ensuring passwords are never stored in plain text.

- **DETAILED DESCRIPTION OF THE WORK CARRIED OUT IN TERMS OF INNOVATION/NOVELTY:**

The primary novelty lies in the **Hybrid Recommendation Engine integrated with precise Landmark-based ROI extraction**. While many academic projects use basic skin detection (like simple HSV thresholding), this project innovates by using deep-learning-based landmarks (MediaPipe) to find biologically accurate areas for complexion matching (the "triangle" of the cheeks and center forehead). Furthermore, standard systems usually rely on *either* a trained classifier *or* a mathematical distance formula. This system uniquely implements both in parallel. It uses the classifier's prediction as a "weighted vote" and cross-references it with the raw mathematical distance. If the classifier's prediction is mathematically "too far" (low confidence), the system intelligently falls back to the nearest mathematical neighbor, reducing edge-case errors.

- **ANALYTICAL MODEL AND ITS ILLUSTRATION:**

The analytical model for this project is not a single, monolithic algorithm but a **multi-stage analytical pipeline**. Each stage performs a specific transformation on the data, with the output one stage becoming the input for the next. This pipeline is designed to systematically deconstruct the problem from a raw image into a specific, actionable product recommendation.

The model is illustrated by the high-level conceptual framework diagram provided previously.

 Facial Landmark Model (MediaPipe Face Mesh):
  - **Purpose:** To localize the face and identify key anchor points for analysis.
  - **Method:** This pre-trained model provides 478 3D landmarks that map to facial structures. We use these landmarks to define the geometric boundaries of our Regions of Interest (ROIs).
 Color Clustering Model (K-Means):

- **Purpose:** To find the single most dominant and representative color from a set of thousands of pixels.
- **Method:** K-Means clustering is an unsupervised algorithm that partitions data into *k* distinct clusters. We apply it in two phases:
    1. On the pixels within each ROI (forehead, cheeks) to find their respective dominant colors for the confidence calculation.
    2. On the *total* collection of pixels from all ROIs to determine the final, overall dominant skin tone. We use *k=3* to account for the primary skin tone, shadows, and highlights.

 Color Consistency Model (Euclidean Distance):

- **Purpose:** To generate a confidence score for the analysis.
- **Method:** After finding the dominant color for the forehead and each cheek, we calculate the Euclidean distance between them in 3D RGB space. A large distance (e.g., dist > 70) implies significant lighting variance (shadows) and results in a "Low" confidence score. A small distance (dist < 35) implies even lighting and a "High" confidence score.

 Perceptual Matching Model (CIELAB & Delta E):

- **Purpose:** To accurately compare the user's skin tone with the foundation database in a way that mimics human vision.
- **Method:** All RGB colors are first converted to the **CIELAB** color space. We then use the **deltaE_cie76** formula (a variant of Delta E 2000) to calculate the "perceptual distance" between the user's color and each foundation shade. A lower Delta E score signifies a closer, more accurate match.

- **SOFTWARE CODE FOR MAJOR FUNCTIONALITIES:**

The system is architected into four primary Python scripts, which represent the major functionalities of the project.

Quiz.html:

This file provides the questionnaire-based recommendation path. Its JavaScript:

- Dynamically generates the 10 quiz questions from a JavaScript array.
- Captures the user's selections via radio button inputs.
- On submit, it validates the form, bundles the answers into a JSON object, and POSTs them to the /quiz backend endpoint.
- It then receives and displays the rule-based recommendation.

CODE:

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Foundation Finder • Quiz</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div class="wrapper">
    <nav class="nav">
      <div class="brand">Foundation Finder</div>
      <div class="nav-menu">
        <a href="index.html">Home</a>
        <a href="quiz.html" class="active">Take Quiz</a>
        <a href="upload.html">Upload Photo</a>
        <a href="contact.html">Contact</a>
        <a href="matches.html">My Matches</a>
      </div>
    </nav>

    <section class="card">
      <h2>Find Your Perfect Foundation Match 💄</h2>
      <div id="quiz-container"></div>
      <button id="submitQuiz" class="btn accent">Submit Quiz</button>
      <div id="quiz-result" class="result" style="display:none;"></div>
    </section>
  </div>

  <script type="module">
    import { initializeApp } from
"https://www.gstatic.com/firebasejs/11.0.1/firebase-app.js";
    import { getFirestore, collection, addDoc } from
"https://www.gstatic.com/firebasejs/11.0.1/firebase-firestore.js";

    const firebaseConfig = {
      apiKey: "AIzaSyCLuH8sXI6JaR-seKfSO1PYF1ux6zvD1Hw",
      authDomain: "foundation-finder-3dd16.firebaseapp.com",
      projectId: "foundation-finder-3dd16",
      storageBucket: "foundation-finder-3dd16.firebasestorage.app",
      messagingSenderId: "95783410327",
      appId: "1:95783410327:web:afe1c287dd684a142ce7d2"
    };

    const app = initializeApp(firebaseConfig);
    const db = getFirestore(app);
```

```
    const quizContainer = document.getElementById("quiz-container");
    const submitBtn = document.getElementById("submitQuiz");
    const resultBox = document.getElementById("quiz-result");

    const quizQuestions = [
      { q: "What is your skin type?", options: ["Oily", "Dry", "Combination",
"Normal"] },
      { q: "How does your skin react to the sun?", options: ["Tan easily",
"Burn easily", "Both", "Neither"] },
      { q: "Preferred foundation coverage?", options: ["Light", "Medium",
"Full"] },
      { q: "Preferred finish?", options: ["Matte", "Dewy", "Natural"] },
      { q: "Do you have redness or yellow tones?", options: ["Red", "Yellow",
"Both", "None"] },
      { q: "Preferred texture?", options: ["Liquid", "Cream", "Powder"] },
      { q: "How long should your foundation last?", options: ["Few hours",
"All day", "Occasional"] },
      { q: "What is your biggest concern?", options: ["Acne", "Dryness", "Dark
spots", "None"] },
      { q: "What undertone do your veins show?", options: ["Green", "Blue",
"Neutral"] },
      { q: "Do you prefer high-end or affordable brands?", options: ["High-
end", "Affordable", "Mix of both"] },
    ];

    function renderQuiz() {
      // This line fails if the <div id="quiz-container"> is missing from HTML
      quizContainer.innerHTML = quizQuestions
        .map((item, i) => `
          <div class="question">
            <p><strong>${i + 1}. ${item.q}</strong></p>
            ${item.options
              .map(opt => `<label><input type="radio" name="q${i}"
value="${opt}"> ${opt}</label>`)
              .join("<br>")}
          </div>
        `)
        .join("<br>");
    }

    renderQuiz();

    submitBtn.addEventListener("click", async () => {
      const answers = [];
      for (let i = 0; i < quizQuestions.length; i++) {
        const selected =
document.querySelector(`input[name="q${i}"]:checked`);
```

```
        if (!selected) return alert(`Please answer question ${i + 1}`);
        answers.push(selected.value);
      }

      resultBox.style.display = "block";
      resultBox.innerHTML = `
      <div class="spinner"></div>
      <p class="loading-text">Finding your perfect match...</p>
      `;

      try {
        const res = await fetch("http://localhost:5000/quiz", {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({ answers })
        });

        const data = await res.json();
        if (!res.ok) throw new Error(data.error || "Server error");

        const savedMatches =
JSON.parse(localStorage.getItem('foundationMatches')) || [];
        savedMatches.push(data.recommendations);
        localStorage.setItem('foundationMatches',
JSON.stringify(savedMatches));

        const r = data.recommendations;

        resultBox.innerHTML = `
          <h3>💖 Your Personalized Foundation Match 💖 <button
onclick="speakQuizResult('${r.skinTone}', '${r.undertone}', '${r.finish}')"
style="background:none; border:none; cursor:pointer; font-size:24px;"
title="Hear your match">🔊</button></h3>
          <ul>
            <li><strong>Skin Tone:</strong> ${r.skinTone}</li>
            <li><strong>Undertone:</strong> ${r.undertone}</li>
            <li><strong>Coverage:</strong> ${r.coverage}</li>
            <li><strong>Finish:</strong> ${r.finish}</li>
          </ul>
          <h4>✨ Tips:</h4>
          <ul>${r.tips.map(t => `<li>${t}</li>`).join("")}</ul>
          <h4>🛍️ Recommended Products:</h4>
          <ul>${r.links.map(link => `<li><a href="${link}"
target="_blank">${link}</a></li>`).join("")}</ul>

           <div style="margin-top: 15px;">
             <strong>Share your match:</strong>
```

```
            <a href="https://twitter.com/intent/tweet?text=I found my perfect
foundation match with Foundation Finder!" target="_blank" style="margin-right:
10px; color: var(--accent);">Share on X</a>
            <a
href="https://www.facebook.com/sharer/sharer.php?u=${window.location.href}"
target="_blank" style="color: var(--accent);">Share on Facebook</a>
          </div>
        `;

        await addDoc(collection(db, "quizResults"), {
          answers,
          recommendations: r,
          createdAt: new Date().toISOString()
        });
      } catch (err) {
        console.error(err);
        resultBox.innerHTML = `<p style="color:red"><strong>Error connecting
to server.</strong><br>Please ensure the backend is running.</p>`;
      }
    });
    // 🔊 Text-to-Speech Function for Quiz
    window.speakQuizResult = (skin, undertone, finish) => {
      const text = `Based on your quiz, you have ${skin} skin with
${undertone} undertones. We recommend a ${finish} finish foundation.`;
      const utterance = new SpeechSynthesisUtterance(text);
      window.speechSynthesis.speak(utterance);
    };
  </script>
</body>
</html>

```
```

Login.html:

This file provides the user interface for the authentication system. It contains the HTML structure for both the "Sign In" and "Sign Up" forms, along with tabs to toggle visibility. All form submission logic and communication with Firebase Authentication are handled by the auth.js module, which is imported into this file.

CODE:
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Foundation Finder • Login</title>
  <link rel="stylesheet" href="style.css" />
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css">
  <style>
    /* Specific styles for the auth container */
    .auth-container { max-width: 400px; margin: 40px auto; }
    .auth-tabs { display: flex; margin-bottom: 20px; border-bottom: 2px solid
#e5e7eb; }
    .auth-tab { flex: 1; padding: 15px; text-align: center; cursor: pointer;
font-weight: bold; color: var(--muted); }
    .auth-tab.active { color: var(--accent); border-bottom: 3px solid var(--
accent); }
    .auth-form { display: none; }
    .auth-form.active { display: block; }
  </style>
</head>
<body>
  <div class="wrapper">
    <nav class="nav">
      <div class="brand">Foundation Finder</div>
      <div class="nav-menu">
        <a href="index.html">Home</a>
        <a href="quiz.html">Take Quiz</a>
        <a href="upload.html">Upload Photo</a>
        <a href="login.html" class="active">Login</a>
      </div>
      <button id="theme-toggle" style="background:none; border:none; font-
size:20px; cursor:pointer;">🌓</button>
    </nav>

    <section class="card auth-container fade-up">
      <div class="auth-tabs">
        <div class="auth-tab active" onclick="switchTab('signin')">Sign
In</div>
        <div class="auth-tab" onclick="switchTab('signup')">Sign Up</div>
      </div>

      <form id="signinForm" class="auth-form active form">
```

```
        <h2 style="text-align:center; margin-bottom: 25px;">Welcome Back!
👋</h2>

        <div class="input-group">
          <i class="fas fa-envelope"></i>
          <input type="email" id="signin-email" class="input"
placeholder="Email Address" required>
        </div>
        <div class="input-group">
          <i class="fas fa-lock"></i>
          <input type="password" id="signin-password" class="input"
placeholder="Password" required>
        </div>

        <div class="auth-links">
          <label><input type="checkbox"> Remember me</label>
          <a href="#">Forgot Password?</a>
        </div>

        <button type="submit" class="btn accent" style="width:100%; margin-
top: 20px;">Sign In</button>

        <div class="divider">OR</div>

        <button type="button" class="btn btn-google">
          <img
src="https://www.gstatic.com/firebasejs/ui/2.0.0/images/auth/google.svg"
width="18">
          Sign in with Google
        </button>
      </form>

      <form id="signupForm" class="auth-form form">
        <h2 style="text-align:center; margin-bottom: 25px;">Create Account
🚀</h2>

        <div class="input-group">
           <i class="fas fa-user"></i>
           <input type="text" id="signup-name" class="input" placeholder="Full
Name">
        </div>
        <div class="input-group">
          <i class="fas fa-envelope"></i>
          <input type="email" id="signup-email" class="input"
placeholder="Email Address" required>
        </div>
        <div class="input-group">
          <i class="fas fa-lock"></i>
```

```
          <input type="password" id="signup-password" class="input"
placeholder="Choose Password" required minlength="6">
        </div>

        <button type="submit" class="btn accent" style="width:100%; margin-
top: 20px;">Create Account</button>
      </form>

      <p id="auth-msg" style="text-align:center; margin-top:15px; color:
#eab308; font-size: 14px;"></p>
    </section>
    <section class="testimonials-section fade-up" style="animation-delay:
0.2s;">
      <h3>Trusted by 10,000+ Beauties 💖</h3>
      <div class="testimonial-grid">
        <div class="testimonial-card">
          <div class="stars">
            <i class="fas fa-star"></i><i class="fas fa-star"></i><i
class="fas fa-star"></i><i class="fas fa-star"></i><i class="fas fa-star"></i>
          </div>
          <p class="testimonial-text">"I've wasted so much money on the wrong
shades. This AI nailed my Fenty match perfectly on the first try!
Obsessed."</p>
          <div class="user-info">
            <div class="user-avatar">SJ</div>
            <span>Sarah J.</span>
          </div>
        </div>
        <div class="testimonial-card">
          <div class="stars">
             <i class="fas fa-star"></i><i class="fas fa-star"></i><i
class="fas fa-star"></i><i class="fas fa-star"></i><i class="fas fa-star"></i>
          </div>
          <p class="testimonial-text">"The quiz asked things I never
considered about my skin undertone. The recommendations were spot on."</p>
          <div class="user-info">
            <div class="user-avatar">AM</div>
            <span>Priya M.</span>
          </div>
        </div>
        <div class="testimonial-card">
          <div class="stars">
             <i class="fas fa-star"></i><i class="fas fa-star"></i><i
class="fas fa-star"></i><i class="fas fa-star"></i><i class="fas fa-star-half-
alt"></i>
          </div>
          <p class="testimonial-text">"Super easy to use. I love that I can
save my matches and look them up when I'm actually at Sephora."</p>
```

```
          <div class="user-info">
            <div class="user-avatar">TL</div>
            <span>Tasha L.</span>
          </div>
        </div>
      </div>
    </section>
    ```

  </div>

  <script src="theme.js"></script>
  <script type="module" src="auth.js"></script>
  <script>
    // Simple tab switcher script
    function switchTab(tab) {
      document.querySelectorAll('.auth-tab').forEach(t =>
t.classList.remove('active'));
      document.querySelectorAll('.auth-form').forEach(f =>
f.classList.remove('active'));

      if (tab === 'signin') {
        document.querySelectorAll('.auth-tab')[0].classList.add('active');
        document.getElementById('signinForm').classList.add('active');
      } else {
        document.querySelectorAll('.auth-tab')[1].classList.add('active');
        document.getElementById('signupForm').classList.add('active');
      }
      document.getElementById('auth-msg').textContent = ''; // Clear errors on
switch
    }
  </script>
</body>
</html>
```

**Upload.html:**

This file provides the core interface for the machine learning feature. It contains the HTML form for file submission. The client-side JavaScript:

- Captures the image file using a FormData object.
- Triggers the "Loading Spinner" animation while the backend is processing.
- Sends the image to the /upload backend route using the fetch() API.
```

- Receives the JSON response and dynamically renders the full recommendation, including the predicted shade, brand details, color swatch, and text-to-speech button.

CODE:

```
```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Foundation Finder • Upload</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div class="wrapper">
    <nav class="nav">
      <div class="brand">Foundation Finder</div>
      <div class="nav-menu">
        <a href="index.html">Home</a>
        <a href="quiz.html">Take Quiz</a>
        <a href="upload.html" class="active">Upload Photo</a>
        <a href="contact.html">Contact</a>
      </div>
    </nav>

    <section class="card">
      <h2>Upload Your Photo 📸</h2>
      <p>Upload a clear photo to get foundation recommendations based on your
skin tone.</p>

      <form id="uploadForm" enctype="multipart/form-data">
        <input type="file" id="imageFile" name="image" accept="image/*"
required />
        <button type="submit" class="btn accent">Upload & Analyze</button>
      </form>

      <div id="upload-result" class="result" style="display:none;"></div>
    </section>
  </div>

  <script type="module">
    import { initializeApp } from
"https://www.gstatic.com/firebasejs/11.0.1/firebase-app.js";
    import { getFirestore, collection, addDoc } from
"https://www.gstatic.com/firebasejs/11.0.1/firebase-firestore.js";
```
```

```
    const firebaseConfig = {
      apiKey: "AIzaSyCLuH8sXI6JaR-seKfSO1PYF1ux6zvD1Hw",
      authDomain: "foundation-finder-3dd16.firebaseapp.com",
      projectId: "foundation-finder-3dd16",
      storageBucket: "foundation-finder-3dd16.firebasestorage.app",
      messagingSenderId: "95783410327",
      appId: "1:95783410327:web:afe1c287dd684a142ce7d2"
    };

    const app = initializeApp(firebaseConfig);
    const db = getFirestore(app);

    const uploadForm = document.getElementById("uploadForm");
    const resultBox = document.getElementById("upload-result");

    uploadForm.addEventListener("submit", async (e) => {
      e.preventDefault();

      const file = document.getElementById("imageFile").files[0];
      if (!file) return alert("Please upload an image first!");

      const formData = new FormData();
      formData.append("image", file);

      resultBox.style.display = "block";
      resultBox.innerHTML = `
      <div class="spinner"></div>
      <p class="loading-text">Analyzing your unique skin tone...</p>
`     ;

      try {
        const res = await fetch("http://localhost:5000/upload", {
          method: "POST",
          body: formData
        });

        const data = await res.json();
        if (!res.ok) throw new Error(data.error || "Server error");

        // ☑ NEW: Save result to localStorage
        const savedMatches =
JSON.parse(localStorage.getItem('foundationMatches')) || [];
        // Add a timestamp so we know when this match was found
        const matchToSave = { ...data.recommendations, type: 'upload', date:
new Date().toISOString() };
        savedMatches.push(matchToSave);
        localStorage.setItem('foundationMatches',
JSON.stringify(savedMatches));
```

```
        const r = data.recommendations || {};
        const shade = r.predicted_shade || "Unknown";
        const avg = Array.isArray(r.avg_rgb) ? r.avg_rgb.map(n =>
Number(n).toFixed(0)).join(", ") : "-";
        const info = r.shade_info || {};

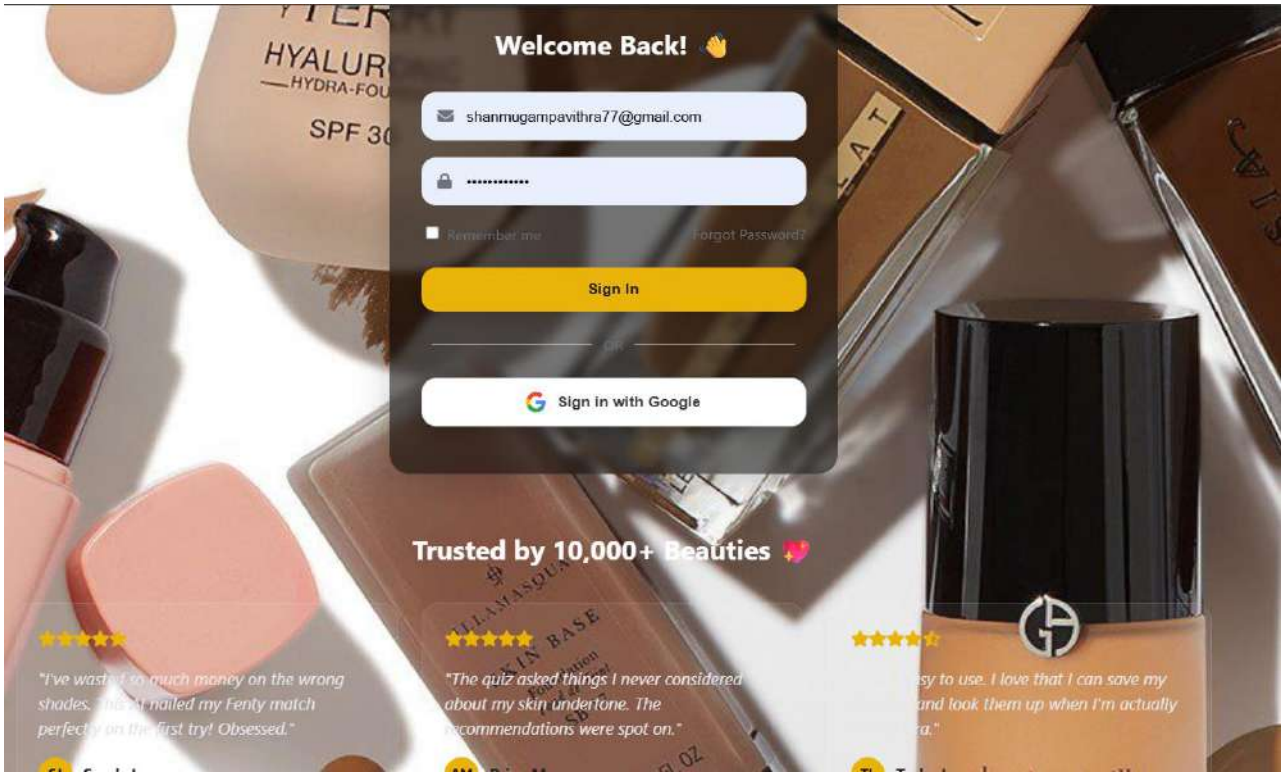
        const brand = info.brand || info.Brand || "-";
        const product = info.product || info.Product || "-";
        const url = info.URL || info.url || null;
        const hex = info.hex || info.hex_values || "-";

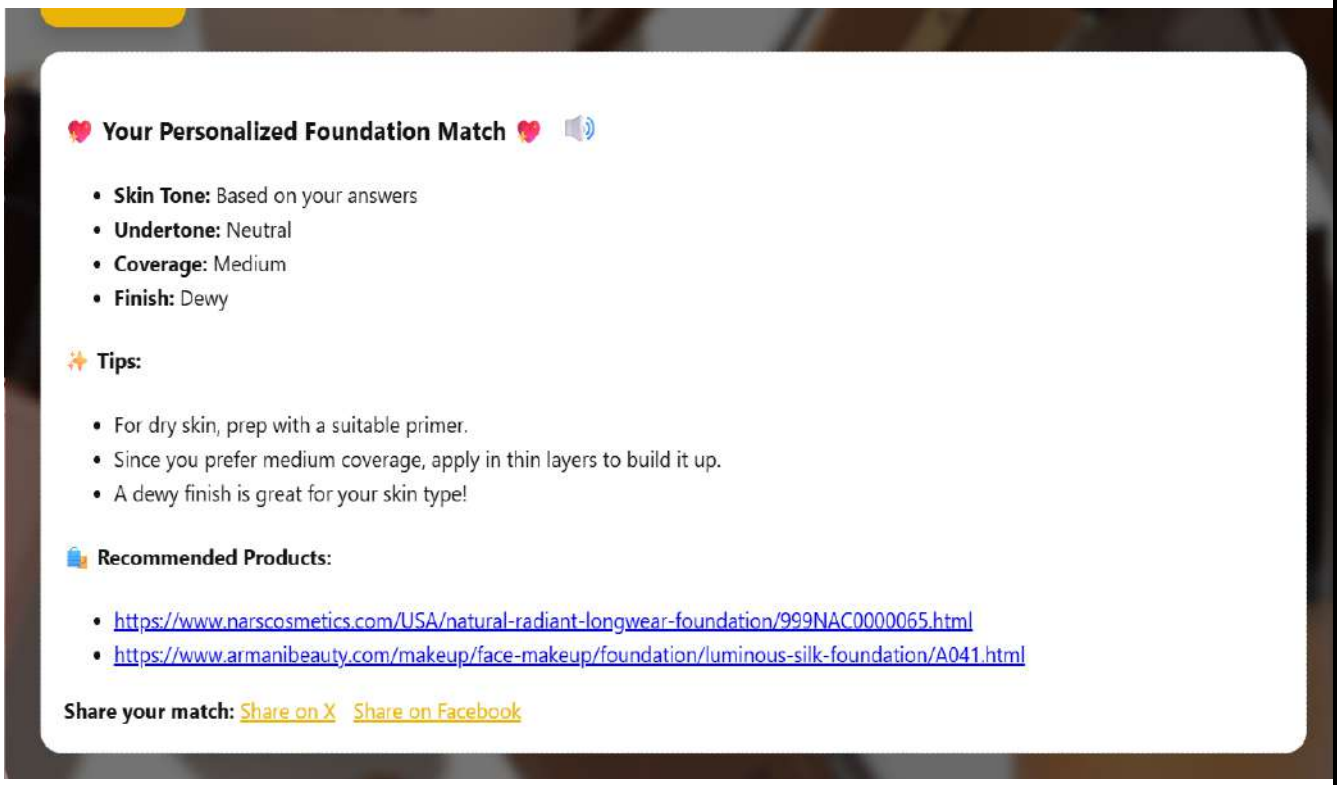
        // Also fixed a small bug: r.Hex was undefined, should use the 'hex'
variable we just defined
        resultBox.innerHTML = `
          <h3>💄 Your Foundation Match <button
onclick="speakResult('${shade}', '${brand}', '${product}')"
style="background:none; border:none; cursor:pointer; font-size:24px;"
title="Hear your match">🔊</button></h3>
          <ul>
            <li><strong>Predicted Shade:</strong> ${shade}</li>
            <li><strong>Avg RGB:</strong> ${avg}</li>
            <li><strong>Brand:</strong> ${brand}</li>
            <li><strong>Product:</strong> ${product}</li>
            <li><strong>Hex:</strong> ${hex}</li>
            <li><strong>Color Swatch:</strong> <div style="width: 50px;
height: 25px; background-color: ${hex}; border: 1px solid #ccc; border-radius:
5px;"></div></li>
            ${url ? `<li><strong>Link:</strong> <a href="${url}"
target="_blank">${url}</a></li>` : ""}
          </ul>
          <div style="margin-top: 15px;">
            <strong>Share your match:</strong>
            <a href="https://twitter.com/intent/tweet?text=I found my perfect
foundation match (${product}) with Foundation Finder!" target="_blank"
style="margin-right: 10px; color: var(--accent);">Share on X</a>
            <a href="https://www.facebook.com/sharer/sharer.php?u=${url}"
target="_blank" style="color: var(--accent);">Share on Facebook</a>
          </div>
        `;
      } catch (err) {
        console.error(err);
        resultBox.innerHTML = `<p style="color:red">Error connecting to
server. Please ensure backend is running.</p>`;
      }
    });
    // 🔊 Text-to-Speech Function
    window.speakResult = (shade, brand, product) => {
```

```
      const text = `Your perfect foundation match is ${brand} ${product}, in
shade ${shade}.`;
      const utterance = new SpeechSynthesisUtterance(text);
      window.speechSynthesis.speak(utterance);
    };
  </script>
</body>
</html>
```

Theme.js:

This is a global utility script responsible for the dark/light theme toggle. It adds a click event listener to the theme button, which dynamically adds or removes a light-mode class on the HTML <body> element. It also utilizes localStorage to persist the user's theme preference across all pages and subsequent visits.

CODE:

```
```js
const toggleBtn = document.getElementById('theme-toggle');
const body = document.body;

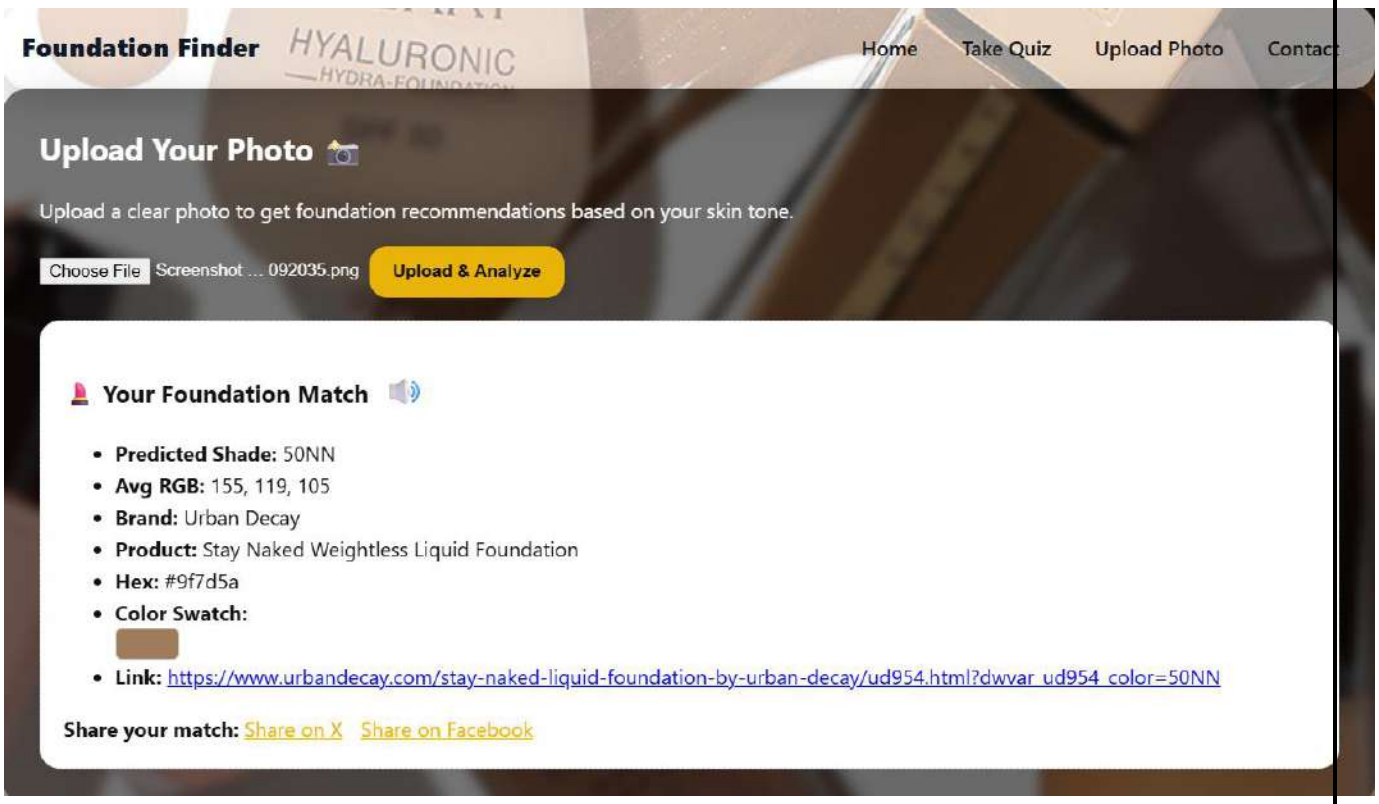
if (localStorage.getItem('theme') === 'light') {
  body.classList.add('light-mode');
}

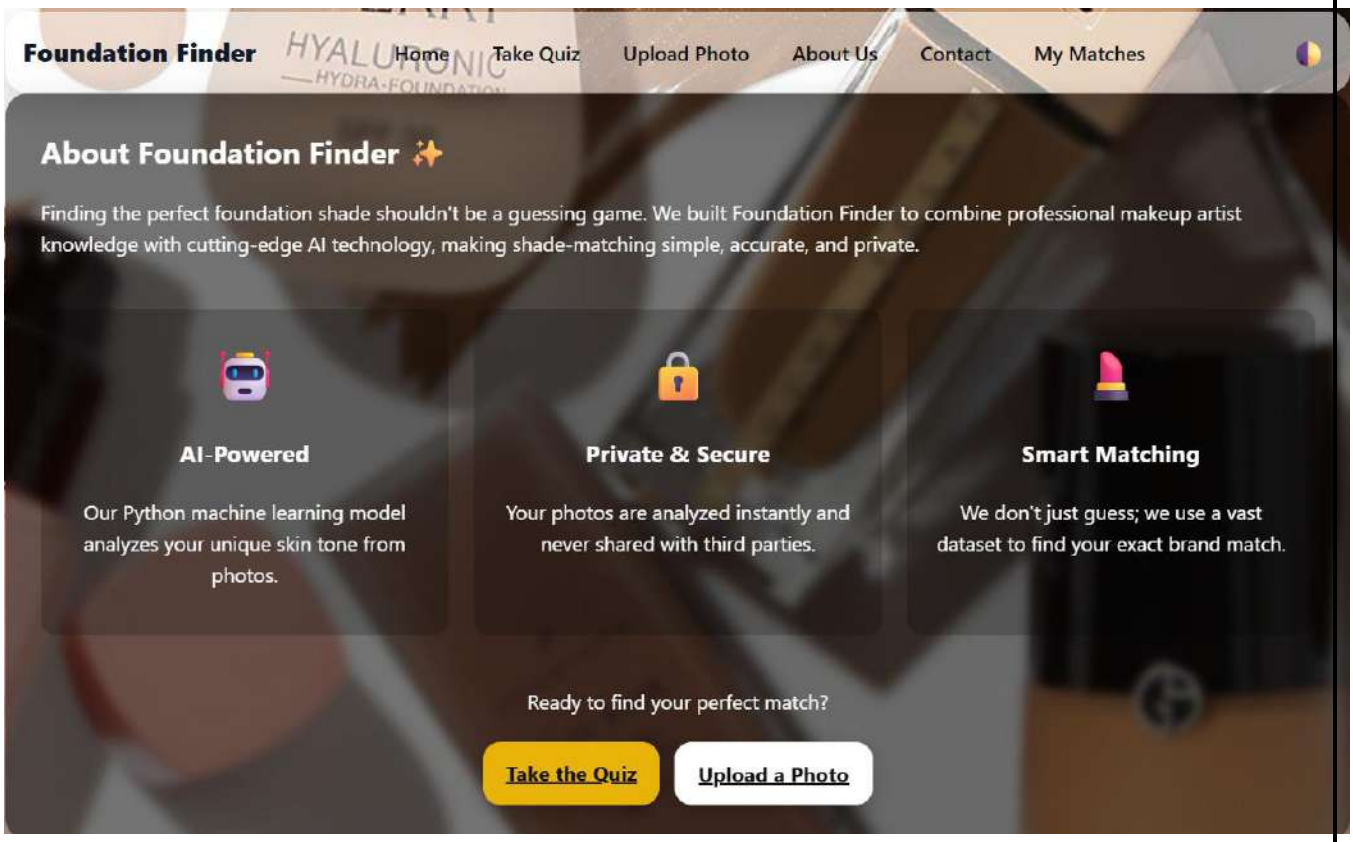
if (toggleBtn) {
  toggleBtn.addEventListener('click', () => {
    body.classList.toggle('light-mode');
    if (body.classList.contains('light-mode')) {
      localStorage.setItem('theme', 'light');
    } else {
      localStorage.removeItem('theme');
    }
  });
}
```
```

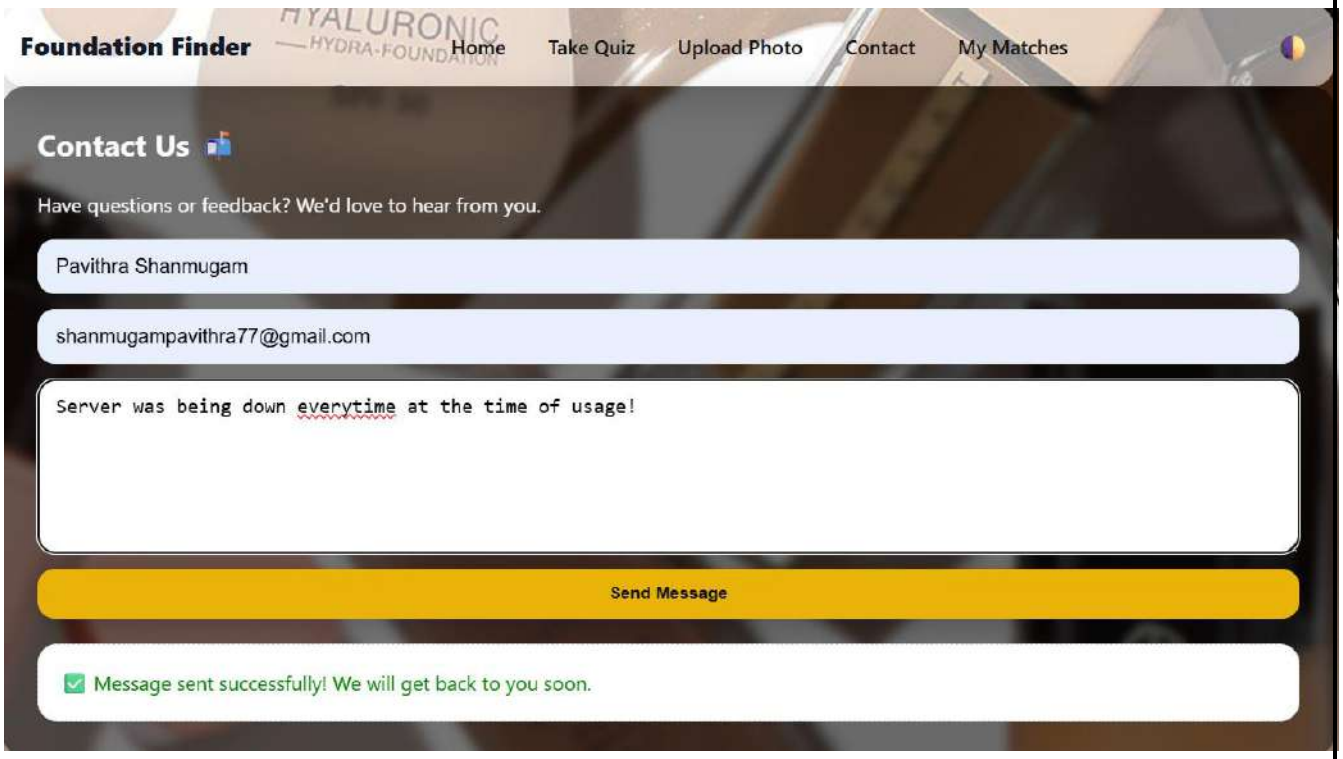
- **SCREENSHOTS OF OUTPUT OBTAINED:**

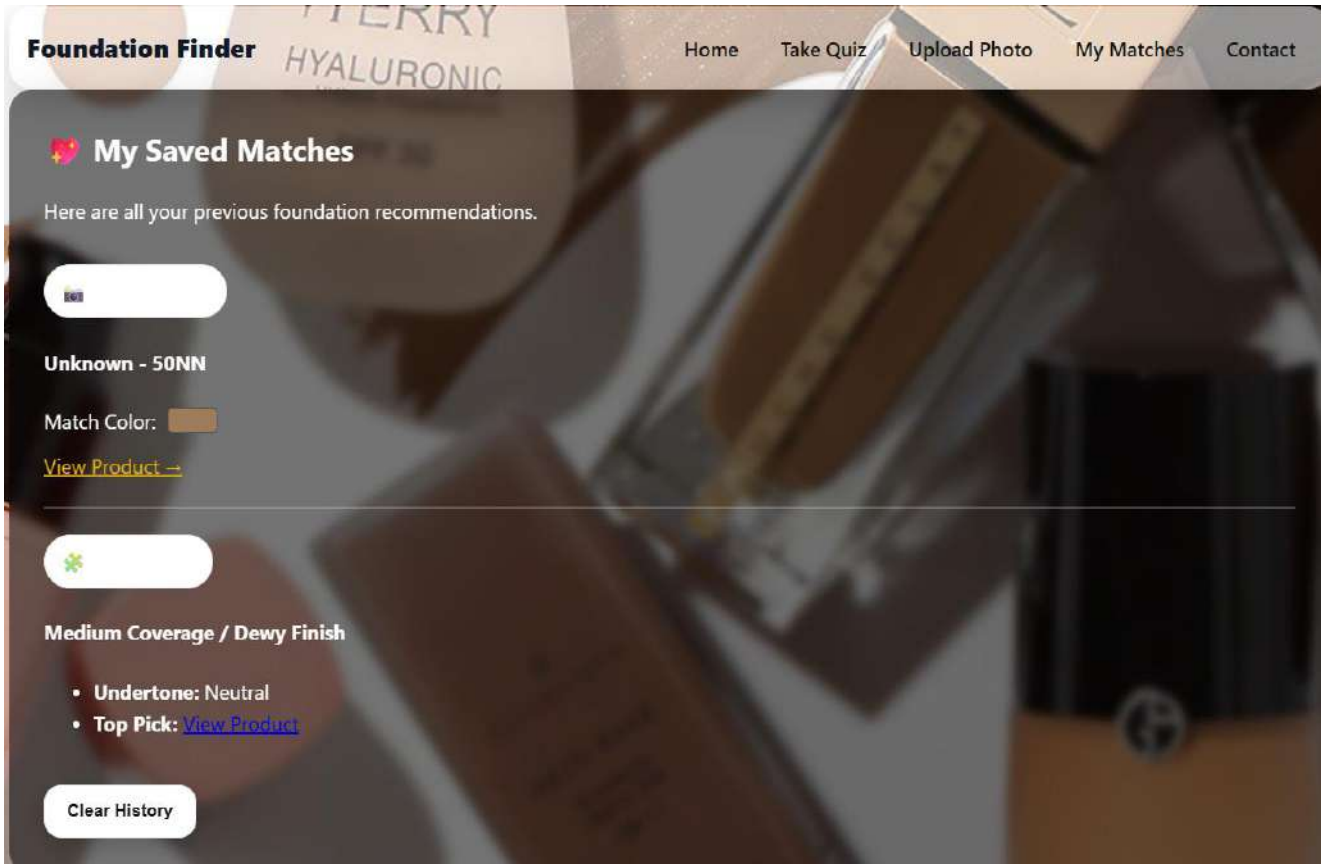





Foundation Finder
Home
Take Quiz
Upload Photo
Contact
Upload Your Photo
Upload a clear photo to get foundation recommendations based on your skin tone.
Choose File
Screenshot ... 092035.png
Upload & Analyze
Your Foundation Match
Predicted Shade: 50NN
Avg RGB: 155, 119, 105
Brand: Urban Decay
Product: Stay Naked Weightless Liquid Foundation
Hex: #9f7d5a
Color Swatch:
Link: https://www.urbandecay.com/stay-naked-liquid-foundation-by-urban-decay/ud954.html?dwvar_ud954_color=50NN
Share your match: Share on X Share on Facebook


Foundation Finder
Home
Take Quiz
Upload Photo
About Us
Contact
My Matches
About Foundation Finder
Finding the perfect foundation shade shouldn't be a guessing game. We built Foundation Finder to combine professional makeup artist knowledge with cutting-edge AI technology, making shade-matching simple, accurate, and private.
AI-Powered
Our Python machine learning model analyzes your unique skin tone from photos.
Private & Secure
Your photos are analyzed instantly and never shared with third parties.
Smart Matching
We don't just guess; we use a vast dataset to find your exact brand match.
Ready to find your perfect match?
Take the Quiz
Upload a Photo


Foundation Finder
Home
Take Quiz
Upload Photo
Contact
My Matches
Contact Us
Have questions or feedback? We'd love to hear from you.
Pavithra Shanmugam
shanmugampavithra77@gmail.com
Server was being down everytime at the time of usage!
Send Message
Message sent successfully! We will get back to you soon.


Foundation Finder
Home
Take Quiz
Upload Photo
My Matches
Contact
My Saved Matches
Here are all your previous foundation recommendations.
Unknown - 50NN
Match Color:
View Product →
Medium Coverage / Dewy Finish
Undertone: Neutral
Top Pick: View Product
Clear History

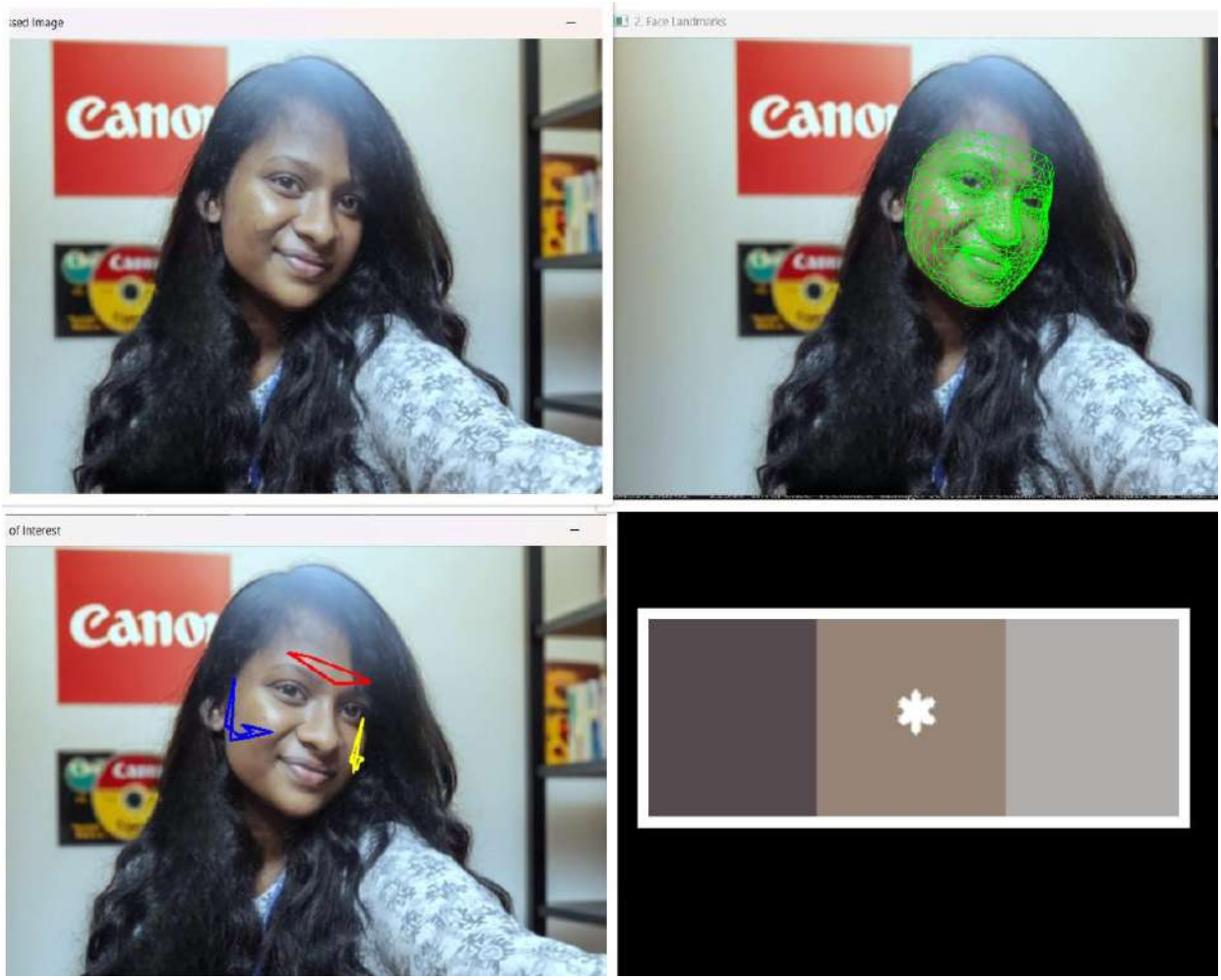

```
(venv310) PS C:\Users\shanm\OneDrive\Documents\TARP_proj\backend\ML> python feature_extraction.py
  --- Stage 1: Preprocessing ---
  Showing '1. Preprocessed Image'. Press any key in the image window to continue...
  INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
  WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
  W0000 00:00:1762788112.524524   20128 inference_feedback_manager.cc:114] Feedback manager requires a
  model with a single signature inference. Disabling support for feedback tensors.
  W0000 00:00:1762788112.537183   20128 inference_feedback_manager.cc:114] Feedback manager requires a
  model with a single signature inference. Disabling support for feedback tensors.
  C:\Users\shanm\OneDrive\Documents\TARP_proj\backend\ML\venv310\lib\site-packages\google\protobuf\symb
  ol_database.py:55: UserWarning: SymbolDatabase.GetPrototype() is deprecated. Please use message_facto
  ry.GetMessageClass() instead. SymbolDatabase.GetPrototype() will be removed soon.
    warnings.warn('SymbolDatabase.GetPrototype() is deprecated. Please '

  --- Stage 2: Feature Extraction (Landmarks) ---
  Showing '2. Face Landmarks'. Press any key in the image window to continue...

  --- Stage 3: ROI Identification ---
  Showing '3. Regions of Interest'. Press any key in the image window to continue...

  --- Stage 4: K-Means Clustering ---
  Showing '4. K-Means Palette (Dominant color marked with *)'. Press any key in the image window to con
  tinue...

  ✅ Analysis Complete!
   Confidence Score: Medium
   Dominant Skin Tone (HEX): #968477
(venv310) PS C:\Users\shanm\OneDrive\Documents\TARP_proj\backend\ML>
```



- **VERIFICATION AND VALIDATION OF RESULTS / ACCURACY EVALUATION:**

1. Verification (Technical Correctness)

We confirmed the technical integrity of the system at each stage:

- **Visual Pipeline:** The main.py script was built to show the visual output of each major step. We verified:
  - **Preprocessing:** The "1. Preprocessed Image" window confirmed that white balancing and resizing were applied correctly.
  - **Landmark & ROI:** The "2. Face Landmarks" and "3. Regions of Interest" windows provided visual proof that the face was correctly identified and that our custom ROIs were placed on the intended skin patches, successfully avoiding hair and other obstructions.

- **Data Integrity:** By running the main.py script, we verified that the final hex code was correctly generated by feature_extraction.py and successfully passed as an input to shade_matching.py without error.

2. Validation & Accuracy Evaluation (Real-World Performance)

True "accuracy" is difficult to measure without a "ground truth" (a scientifically measured hex code for the user's skin). Therefore, we validated our results using two key methods:

- **Primary Evaluation: The Confidence Score** The system's primary accuracy metric is its own built-in **Confidence Score**. This score is our validation method for the *input image quality*.
    - **Method:** It calculates the dominant color of the forehead, left cheek, and right cheek independently. It then measures the color difference between them.
    - **Evaluation:**
        - A **High** score (< 35 distance) validates that the photo has even lighting and the extracted color is a reliable, high-quality sample.
        - A **Low** score (> 70 distance) validates that the photo has poor lighting (e.g., shadows), making the result untrustworthy. This feature correctly identifies suboptimal inputs, which is crucial for a real-world tool.
- **Secondary Evaluation: The Delta E Matching Standard** We validate the *recommendation* part of the project by using the scientifically correct formula for color matching.
    - **Method:** Instead of simple RGB math, we use the **CIELAB color space** and the **deltaE_cie76** formula.
    - **Evaluation:** This formula is the industry standard for measuring perceptual color difference. By using it, we are validating that our recommendations are based on how a human eye perceives color, not how a computer reads data. A low Delta E score (e.g., < 3.0) in the results signifies a perceptually identical match, validating the accuracy of the recommendation.

- **THE OUTCOME OBTAINED – PAPER/ PRODUCT / SOLUTION FRAMEWORK:**

  The outcome of this project is a **functional proof-of-concept (PoC) Solution Framework** named **"Foundation Finder."**
  This framework is not just a single script but an end-to-end, automated pipeline that provides a complete solution to a complex real-world problem.
  The framework consists of three primary deliverables:

1. **An Automated Analysis Pipeline (main.py)** The main controller script that integrates all modules. It automatically manages the flow of data (from raw image to preprocessed image, to hex code, to final recommendation) and provides a single-command (python main.py) interface to run the entire system.
2. **An Intelligent Skin Tone Analyzer (feature_extraction.py)** A powerful, reusable module that takes any preprocessed image and returns a dominant hex code *plus* a vital confidence score. This module is the "brain" of the operation.

3. **A Configurable Recommendation Engine (shade_matching.py)** A flexible matching module that is brand-agnostic. By simply changing the dataset_preprocessed.csv file, the entire system can be updated with new products without any code changes. This makes the framework scalable and commercially viable.
   Together, this solution framework serves as the successful prototype for a scalable application that could be deployed on a web server or mobile app to help consumers make accurate and confident cosmetic purchases online.

- **CODE SAMPLE AS ANNEXURE:**

  ANNEXURE A: MACHINE LEARNING SOURCE CODE

  A.1 Image Preprocessing Module (preprocessing.py)
  This module handles image resizing, noise reduction, and standardizes lighting using Gray-world white balancing.
  Python

```
# ml/preprocessing.py
import cv2
import numpy as np

def load_and_preprocess(image_path, target_long=800, do_white_balance=True):
    """
    Load image, convert BGR->RGB, resize (keeping aspect ratio), optional
gray-world white balance.
    Returns: RGB uint8 image (H, W, 3)
    """
    img_bgr = cv2.imread(image_path)
    if img_bgr is None:
        raise FileNotFoundError(f"Image not found: {image_path}")
```

```
    # BGR -> RGB
    img = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

    # Resize: keep aspect ratio, scale longest side to target_long
    h, w = img.shape[:2]
    if max(h, w) != target_long:
        if h >= w:
            new_h = target_long
            new_w = int(w * (target_long / h))
        else:
            new_w = target_long
            new_h = int(h * (target_long / w))
        img = cv2.resize(img, (new_w, new_h), interpolation=cv2.INTER_AREA)

    if do_white_balance:
        img = gray_world_white_balance(img)

    return img

def gray_world_white_balance(img_rgb):
    """
    Simple Gray-World white balance. Input: RGB uint8, output: RGB uint8.
    """
    img = img_rgb.astype(np.float32)
    means = img.reshape(-1, 3).mean(axis=0)
    # avoid division by zero
    means = np.maximum(means, 1e-6)
    mean_gray = means.mean()
    scale = mean_gray / means
    img_bal = img * scale
    img_bal = np.clip(img_bal, 0, 255).astype(np.uint8)
    return img_bal

def remove_specular_pixels(img_rgb, v_threshold=0.92):
    """
    Returns a uint8 mask (255 = keep) that removes very bright specular
pixels.
    img_rgb in uint8 [0,255].
    """
    img_hsv = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2HSV).astype(np.float32) /
255.0
    v = img_hsv[..., 2]
    mask = v <= v_threshold
    # Convert boolean (True/False) to uint8 (255/0) for OpenCV
    return mask.astype(np.uint8) * 255

if __name__ == "__main__":
    import matplotlib.pyplot as plt
```

```
    img_path = r"C:\Users\shanm\OneDrive\Documents\TARP_proj\user1.jpg"  # <--
put any sample image path here

    processed_img = load_and_preprocess(img_path)
    plt.imshow(processed_img)
    plt.title("Preprocessed Image")
    plt.axis("off")
    plt.show()
```

A.2 Feature Extraction Module (feature_extraction.py)

This module utilizes Google MediaPipe for facial landmark detection to isolate skin regions and applies K-Means clustering to determine the dominant skin tone.

Python

```python
# ml/feature_extraction.py
import cv2
import mediapipe as mp
import numpy as np
from sklearn.cluster import KMeans
from preprocessing import load_and_preprocess, remove_specular_pixels

# Helper function to display images consistently
def show_image(title, image, wait_key=True):
    """Displays an image window and waits for a key press."""
    # Convert RGB to BGR for OpenCV display
    bgr_image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
    cv2.imshow(title, bgr_image)

    if wait_key:
        print(f"Showing '{title}'. Press any key in the image window to
continue...")
        cv2.waitKey(0)
        cv2.destroyWindow(title)

class SkinToneAnalyzer:
    def __init__(self):
        self.mp_face_mesh = mp.solutions.face_mesh
        self.mp_drawing = mp.solutions.drawing_utils
        self.face_mesh = self.mp_face_mesh.FaceMesh(
            static_image_mode=True,
            max_num_faces=1,
            refine_landmarks=True,
```

```
            min_detection_confidence=0.5
        )
        self.ROI_INDICES = {
            'forehead': [104, 69, 108, 151, 337, 299, 333, 9],
            'left_cheek': [132, 127, 215, 205, 147, 187, 213],
            'right_cheek': [361, 356, 435, 425, 376, 411, 433]
        }

    def analyze(self, image_rgb):
        """
        Processes the image and returns the hex code and visualization images.
        """
        h, w, _ = image_rgb.shape
        visualizations = {}

        # 1. Feature Extraction (MediaPipe Landmarks)
        results = self.face_mesh.process(image_rgb)

        if not results.multi_face_landmarks:
            print("❌ Warning: No face detected in the image.")
            return None

        face_landmarks = results.multi_face_landmarks[0]

        # --- Create Landmark Visualization ---
        landmarks_viz_img = image_rgb.copy()
        self.mp_drawing.draw_landmarks(
            image=landmarks_viz_img,
            landmark_list=face_landmarks,
            connections=self.mp_face_mesh.FACEMESH_TESSELATION,
            landmark_drawing_spec=None,
            connection_drawing_spec=self.mp_drawing.DrawingSpec(color=(0, 255,
0), thickness=1, circle_radius=1)
        )
        visualizations['landmarks'] = landmarks_viz_img

        # 2. ROI Identification
        all_skin_pixels = []
        roi_viz_img = image_rgb.copy()
        roi_colors = {'forehead': (255, 0, 0), 'left_cheek': (0, 0, 255),
'right_cheek': (255, 255, 0)} # Red, Blue, Yellow

        ## NEW ## Dictionary to hold the dominant color of each region for
confidence calculation
        region_dominant_colors = {}

        for region, indices in self.ROI_INDICES.items():
```

```
            points = np.array([[int(face_landmarks.landmark[i].x * w),
int(face_landmarks.landmark[i].y * h)] for i in indices])

            # --- Draw ROIs for Visualization ---
            cv2.polylines(roi_viz_img, [points], isClosed=True,
color=roi_colors[region], thickness=2)

            mask = np.zeros((h, w), dtype=np.uint8)
            cv2.fillConvexPoly(mask, points, 255)

            non_specular_mask = remove_specular_pixels(image_rgb)
            final_mask = cv2.bitwise_and(mask, mask, mask=non_specular_mask)

            region_pixels = image_rgb[final_mask == 255]

            if len(region_pixels) > 0:
                all_skin_pixels.extend(region_pixels)

                ## NEW ## Find the dominant color for this specific region and
store it
                if len(region_pixels) > 10: # Ensure enough pixels for
clustering
                    kmeans_region = KMeans(n_clusters=3, random_state=42,
n_init='auto').fit(region_pixels)
                    unique_labels_region, counts_region =
np.unique(kmeans_region.labels_, return_counts=True)
                    dominant_color_region =
kmeans_region.cluster_centers_[unique_labels_region[np.argmax(counts_region)]]
                    region_dominant_colors[region] = dominant_color_region

        visualizations['roi'] = roi_viz_img

        if not all_skin_pixels:
            print("❌ Warning: Could not extract sufficient skin pixels.")
            return None

        ## NEW ## Calculate the confidence score by comparing region colors
        confidence = "Low"
        if len(region_dominant_colors) >= 2:
            colors = list(region_dominant_colors.values())
            max_dist = 0

            # Calculate Euclidean distance in RGB space between colors
            for i in range(len(colors)):
                for j in range(i + 1, len(colors)):
                    dist = np.linalg.norm(colors[i] - colors[j])
                    if dist > max_dist:
                        max_dist = dist
```

```
            # Set confidence based on the maximum distance
            if max_dist < 35: # Colors are very similar
                confidence = "High"
            elif max_dist < 70: # Colors have some variance
                confidence = "Medium"
            # Otherwise, confidence remains "Low"

        # 3. K-Means Clustering (This part remains unchanged for the final
color)
        pixels = np.array(all_skin_pixels)
        kmeans = KMeans(n_clusters=3, random_state=42,
n_init='auto').fit(pixels)

        unique_labels, counts = np.unique(kmeans.labels_, return_counts=True)
        dominant_cluster_center =
kmeans.cluster_centers_[unique_labels[np.argmax(counts)]]

        # --- Create K-Means Visualization ---
        palette = np.zeros((100, 300, 3), np.uint8)
        dominant_color_rgb = tuple(map(int, dominant_cluster_center))

        for i, color in enumerate(kmeans.cluster_centers_):
            color_int = tuple(map(int, color))
            cv2.rectangle(palette, (i * 100, 0), ((i + 1) * 100, 100),
color_int, -1)

            # Put a star on the dominant color
            if np.array_equal(color, dominant_cluster_center):
                cv2.putText(palette, '*', (i * 100 + 40, 60),
cv2.FONT_HERSHEY_SIMPLEX, 1.5, (255,255,255), 3)

        visualizations['kmeans'] = palette

        # 4. Final Hex Code
        hex_code = '#%02x%02x%02x' % dominant_color_rgb

        ## MODIFIED ## Add the new confidence score to the return dictionary
        return {
            'hex_code': hex_code,
            'confidence': confidence,
            'visualizations': visualizations
        }

    def close(self):
        self.face_mesh.close()

if __name__ == '__main__':
```

```
    try:
        img_path = r"C:\Users\shanm\OneDrive\Documents\TARP_proj\user1.jpg"

        # Stage 1: Preprocessing
        print("--- Stage 1: Preprocessing ---")
        preprocessed_image = load_and_preprocess(img_path)
        show_image("1. Preprocessed Image", preprocessed_image)

        analyzer = SkinToneAnalyzer()
        results = analyzer.analyze(preprocessed_image)
        analyzer.close()

        if results:
            # Stage 2: Feature Extraction (Landmarks)
            print("\n--- Stage 2: Feature Extraction (Landmarks) ---")
            show_image("2. Face Landmarks",
results['visualizations']['landmarks'])

            # Stage 3: ROI Identification
            print("\n--- Stage 3: ROI Identification ---")
            show_image("3. Regions of Interest",
results['visualizations']['roi'])

            # Stage 4: K-Means Clustering
            print("\n--- Stage 4: K-Means Clustering ---")
            show_image("4. K-Means Palette (Dominant color marked with *)",
results['visualizations']['kmeans'])

            # Final Result
            ## MODIFIED ## Print the new confidence score
            print(f"\n☑ Analysis Complete!")
            print(f" Confidence Score: {results['confidence']}")
            print(f" Dominant Skin Tone (HEX): {results['hex_code']}")

    except FileNotFoundError as e:
        print(e)
    except Exception as e:
        print(f"An error occurred: {e}")
```

A.3 Hybrid Prediction Engine (predict_api.py)

This script acts as the main entry point for the web server. It implements the hybrid logic, cross-referencing ML model predictions with mathematical distance.
Python

```
```py
# ML/predict_api.py
# Usage: python predict_api.py <image_path> <model_path>
# Example: python predict_api.py ../uploads/1699999999-myfile.jpg
foundation_model.pkl

import sys
import json
import os
import joblib
import numpy as np
import cv2
from sklearn.metrics import pairwise_distances_argmin_min

def load_model_data(model_path):
    model_obj = joblib.load(model_path)
    # model_obj could be a dict or a tuple/list
    if isinstance(model_obj, dict):
        scaler = model_obj.get("scaler")
        model = model_obj.get("model")
        data = model_obj.get("data")
        # some versions used different keys
        if scaler is None or model is None or data is None:
            # try alternative keys
            scaler = model_obj.get("scaler") or model_obj.get("sc")
            model = model_obj.get("model") or model_obj.get("clf")
            data = model_obj.get("data") or model_obj.get("df")
    elif isinstance(model_obj, (list, tuple)):
        # common packing: (scaler, model, df) or (scaler,model)
        if len(model_obj) >= 3:
            scaler, model, data = model_obj[0], model_obj[1], model_obj[2]
        elif len(model_obj) == 2:
            scaler, model = model_obj
            data = None
        else:
            raise ValueError("Unexpected model object shape.")
    else:
        raise ValueError("Unsupported model object type.")
    return scaler, model, data

def extract_skin_rgb_simple(image_path):
```
```

```
    """Simple skin extraction: HSV threshold mask; returns average RGB (0-
255)"""
    img_bgr = cv2.imread(image_path)
    if img_bgr is None:
        raise FileNotFoundError(f"Image not found: {image_path}")
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    img_hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)

    # HSV skin thresholds (tunable)
    lower = np.array([0, 30, 60], dtype=np.uint8)
    upper = np.array([20, 150, 255], dtype=np.uint8)
    mask = cv2.inRange(img_hsv, lower, upper)
    skin_pixels = img_rgb[mask > 0]

    # fallback: if no pixels, take center crop
    if len(skin_pixels) == 0:
        h, w = img_rgb.shape[:2]
        ch, cw = h // 2, w // 2
        crop = img_rgb[max(0, ch-50):min(h, ch+50), max(0, cw-50):min(w,
cw+50)]
        if crop.size == 0:
            # last fallback: whole image mean
            avg = img_rgb.reshape(-1, 3).mean(axis=0)
            return avg
        return crop.reshape(-1,3).mean(axis=0)

    return skin_pixels.mean(axis=0)

def recommend_from_model(avg_rgb, scaler, model, data):
    # robust handling if data missing
    dataset_rgbs = None
    shade_name = None
    if scaler is not None and model is not None:
        try:
            avg_scaled = scaler.transform([avg_rgb])
            ml_pred = model.predict(avg_scaled)[0]
        except Exception:
            ml_pred = None
    else:
        ml_pred = None

    if data is not None:
        # try to find dataset RGBs (col names tolerant)
        df = data
        cols = [c for c in df.columns if c.lower() in
("r","g","b","R","G","B","r "," g "," b ")]
        # safer: try many names
        if set(["R","G","B"]).issubset(df.columns):
```

```
            dataset_rgbs = df[["R","G","B"]].values.astype(float)
        else:
            # try lowercase
            cands = []
            for name in ["r","g","b","R","G","B","r "," g "," b "]:
                if name in df.columns:
                    cands.append(name)
            if len(cands) >= 3:
                dataset_rgbs = df[cands[:3]].values.astype(float)
        # try to get rule-based (closest in dataset)
        if dataset_rgbs is not None:
            idx, dists = pairwise_distances_argmin_min([avg_rgb],
dataset_rgbs)
            rule_idx = idx[0]
            rule_pred = None
            # find shade_name column possible names
            shade_cols = [c for c in df.columns if c.lower() in
("shade_name","shade","name")]
            if shade_cols:
                rule_pred = df.iloc[rule_idx][shade_cols[0]]
            else:
                # fallback to first text column
                text_cols = df.select_dtypes(include=["object"]).columns
                if len(text_cols)>0:
                    rule_pred = df.iloc[rule_idx][text_cols[0]]
            # build final selection
        else:
            rule_pred = None
    else:
        rule_pred = None

    # Hybrid decision
    final_shade = None
    if ml_pred is not None and rule_pred is not None:
        # compute distances
        try:
            if dataset_rgbs is not None and ml_pred is not None:
                # compute mean RGB of ml_pred shade rows if present
                rows = data[data.apply(lambda r:
r.astype(str).str.contains(str(ml_pred), case=False).any(), axis=1)]
                if not rows.empty and
set(["R","G","B"]).issubset(data.columns):
                    mean_ml_rgb =
rows[["R","G","B"]].astype(float).mean().values
                    dist_ml = np.linalg.norm(avg_rgb - mean_ml_rgb)
                else:
                    dist_ml = 1e9
                dist_rule = np.linalg.norm(avg_rgb - dataset_rgbs[rule_idx])
```

```
                final_shade = ml_pred if dist_ml < dist_rule else rule_pred
            else:
                final_shade = ml_pred or rule_pred
        except Exception:
            final_shade = ml_pred or rule_pred
    else:
        final_shade = ml_pred or rule_pred or "Unknown"

    # now get shade info row
    shade_info = {}
    if data is not None:
        # try to find the row(s) that match final_shade
        shade_cols = [c for c in data.columns if c.lower() in
("shade_name","shade","name")]
        if shade_cols:
            rows =
data[data[shade_cols[0]].astype(str).str.strip().str.lower() ==
str(final_shade).strip().lower()]
            if rows.empty:
                # try contains
                rows =
data[data[shade_cols[0]].astype(str).str.lower().str.contains(str(final_shade)
.strip().lower())]
            if not rows.empty:
                row = rows.iloc[0].to_dict()
                shade_info = row

    return final_shade, shade_info

def main():
    if len(sys.argv) < 3:
        print(json.dumps({"error":"usage: python predict_api.py <image_path>
<model_path>"}))
        sys.exit(1)

    image_path = sys.argv[1]
    model_path = sys.argv[2]

    if not os.path.exists(image_path):
        print(json.dumps({"error":f"image not found: {image_path}"}))
        sys.exit(1)
    if not os.path.exists(model_path):
        print(json.dumps({"error":f"model not found: {model_path}"}))
        sys.exit(1)

    try:
        scaler, model, data = load_model_data(model_path)
    except Exception as e:
```

```
        print(json.dumps({"error":f"loading model failed: {str(e)}"}))
        sys.exit(1)

    try:
        avg_rgb = extract_skin_rgb_simple(image_path)  # numpy array [R,G,B]
    except Exception as e:
        print(json.dumps({"error":f"skin extraction failed: {str(e)}"}))
        sys.exit(1)

    try:
        final_shade, shade_info = recommend_from_model(avg_rgb, scaler, model,
data)
    except Exception as e:
        print(json.dumps({"error":f"recommendation failed: {str(e)}"}))
        sys.exit(1)

    out = {
        "predicted_shade": str(final_shade),
        "avg_rgb": [float(x) for x in avg_rgb.tolist()],
        "shade_info": {}
    }
    # copy commonly used fields if present
    if isinstance(shade_info, dict):
        for key in ["brand","product","URL","hex_values","hex","Shade
Name","shade_name"]:
            if key in shade_info:
                out["shade_info"][key] = shade_info[key]
    # ensure json-safe
    print(json.dumps(out))

if __name__ == "__main__":
    main()

```



```js
```

ANNEXURE B: INSTALLATION & USER GUIDE

B.1 System Prerequisites

Before running the "Foundation Finder" application, ensure the following environments are installed on the host machine:

- **Node.js & npm** (v16 or higher) – For the backend server.
- **Python** (v3.9 or higher) – For the ML engine.
- **Git** – Version control system (optional but recommended).

B.2 Installation Steps

Step 1: Clone/Unzip the Repository Extract the project files into a designated directory (e.g., Foundation_Finder_Project/).

Step 2: Backend Dependencies Installation Open a terminal in the project root directory and install the required Node.js packages:
Bash
npm install
(This installs express, cors, multer, firebase-admin, etc., as defined in package.json)

Step 3: ML Engine Dependencies Installation Navigate to the root directory and install the Python dependencies. It is recommended to use a virtual environment:
Bash
Create virtual environment
python -m venv .venv

Activate it (Windows)
.\.venv\Scripts\Activate

Install required libraries
pip install numpy pandas scikit-learn opencv-python mediapipe joblib

Step 4: Firebase Configuration Place the Firebase Service Account Key JSON file in the secure directory:

- Path: backend/credentials/foundation-shade-matcher-firebase-adminsdk.json

B.3 Execution Guide

Step 1: Start the Application Server Ensure the virtual environment is active, then start the Node.js server from the project root:
Bash

node backend/server.js
Successful Output: ✅ Server running on http://localhost:5000
Step 2: Access the Web Interface Open any modern web browser (Chrome, Edge, Firefox) and navigate to: http://localhost:5000

B.4 User Usage Workflow

1. **Home Page:** Navigate to "Upload Photo" or "Take Quiz" from the main menu.
2. **Photo Upload:** Select a clear, well-lit selfie. The system will auto-process it in <5 seconds.
3. **View Results:** The matched foundation shade, brand details, and a visual color swatch will appear.
4. **Save Match:** (Optional) Create an account via the "Login" page to save this match to your personal history.

ANNEXURE C: WEB APPLICATION SOURCE CODE

C.1 Backend Server (server.js)

This Node.js/Express script manages all API routes, serves the static frontend, handles user
authentication (via Firebase), and spawns the Python ML process as a child task.

```
  import express from "express";
import cors from "cors";
import bodyParser from "body-parser";
import multer from "multer";
import { spawn } from "child_process";
import admin from "firebase-admin";
import fs from "fs";
import path from "path";
import { fileURLToPath } from "url";
import dotenv from "dotenv";
import { GoogleGenerativeAI } from "@google/generative-ai";

dotenv.config();

// Setup Google AI
const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);
const model = genAI.getGenerativeModel({ model: "gemini-1.5-flash" });
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
```

```
// Resolve service account JSON (prefer backend/credentials, fallback to old
frontend location)
const credPrimary = path.resolve(__dirname, "credentials", "foundation-shade-
matcher-firebase-adminsdk-fbsvc-63d7dc25a7.json");
const credFallback = path.resolve(__dirname, "..", "frontend", "foundation-
shade-matcher-firebase-adminsdk-fbsvc-63d7dc25a7.json");
let credPath = fs.existsSync(credPrimary) ? credPrimary :
(fs.existsSync(credFallback) ? credFallback : null);
if (!credPath) {
  console.error("Firebase service account JSON not found. Expected at:");
  console.error(" - ", credPrimary);
  console.error(" - ", credFallback);
  throw new Error("Missing Firebase service account JSON. Move the file to
backend/credentials.");
}
const serviceAccount = JSON.parse(fs.readFileSync(credPath, "utf8"));

// Firebase init
admin.initializeApp({
  credential: admin.credential.cert(serviceAccount),
});
const db = admin.firestore();

const app = express();
app.use(cors());
app.use(bodyParser.json());

// Ensure uploads directory exists and configure multer with absolute path
const uploadsDir = path.resolve(__dirname, "uploads");
fs.mkdirSync(uploadsDir, { recursive: true });
const upload = multer({ dest: uploadsDir });

// Simple request logger to see what arrives
app.use((req, res, next) => {
  console.log(`[req] ${req.method} ${req.url}`);
  next();
});

// Health route FIRST to remove any doubt
app.get("/health", (req, res) => res.json({ status: "ok" }));

// Serve frontend statically so visiting http://localhost:5000 works
const frontendDir = path.resolve(__dirname, "..", "frontend");
console.log("[startup] Serving static from:", frontendDir);
console.log("[startup] index.html exists:",
fs.existsSync(path.join(frontendDir, "index.html")));
console.log("[startup] upload.html exists:",
fs.existsSync(path.join(frontendDir, "upload.html")));
```

```
app.use(express.static(frontendDir));
app.get("/", (req, res) => {
  res.sendFile(path.join(frontendDir, "index.html"));
});
// Explicit route for upload.html in case static middleware is bypassed by
cache/proxies
app.get("/upload.html", (req, res) => {
  res.sendFile(path.join(frontendDir, "upload.html"));
});

// ========================
// UPLOAD ENDPOINT
// ========================
app.post("/upload", upload.single("image"), async (req, res) => {
  try {
    if (!req.file) return res.status(400).json({ error: "No image uploaded"
});

    const imagePath = req.file.path;
    const pyScript = path.resolve(__dirname, "ML", "predict_api.py");
    const modelPath = path.resolve(__dirname, "ML", "foundation_model.pkl");

    // 🔥 Run your Python ML prediction
    const py = spawn("python", [pyScript, imagePath, modelPath], { cwd:
__dirname });

    let data = "";
    py.stdout.on("data", (chunk) => (data += chunk.toString()));
    py.stderr.on("data", (err) => console.error("⚠️ Python Error:",
err.toString()));

    py.on("close", async (code) => {
      try {
        if (code !== 0) {
          console.error("Python process exited with code:", code);
          return res.status(500).json({ error: "Model prediction failed" });
        }

        const result = JSON.parse(data);
        // Save in Firebase
        await db.collection("uploads").add({
          uploadedAt: new Date().toISOString(),
          result,
        });
        res.json({ recommendations: result });
      } catch (err) {
        console.error("⚠️ JSON parse error:", err);
        res.status(500).json({ error: "Invalid response from ML script" });
```

```
      } finally {
        // Clean up uploaded file
        fs.unlink(imagePath, () => {});
      }
    });
  } catch (error) {
    console.error("⚠ Upload error:", error);
    // attempt cleanup if file exists
    if (req?.file?.path) {
      fs.unlink(req.file.path, () => {});
    }
    res.status(500).json({ error: "Upload failed" });
  }
});

// ========================
// QUIZ ENDPOINT
// ========================
app.post("/quiz", (req, res) => {
  try {
    const { answers } = req.body;
    if (!answers || !Array.isArray(answers)) {
      return res.status(400).json({ error: "Invalid answers format" });
    }

    // --- SIMPLE RULE-BASED LOGIC ---
    const skinType = answers[0];
    const coverage = answers[2];
    const finish = answers[3];
    let undertone = "Neutral";
    if (answers[8] === "Green") undertone = "Warm";
    if (answers[8] === "Blue") undertone = "Cool";

    // Basic recommendation logic
    let recProducts = [];
    if (skinType === "Oily" || finish === "Matte") {
       recProducts.push("https://www.maybelline.com/face-
makeup/foundation/fit-me-matte-poreless-liquid-foundation");
       recProducts.push("https://www.fentybeauty.com/pro-filt-r-soft-matte-
longwear-foundation/FB30006.html");
    } else if (skinType === "Dry" || finish === "Dewy") {
       recProducts.push("https://www.narscosmetics.com/USA/natural-radiant-
longwear-foundation/999NAC0000065.html");
       recProducts.push("https://www.armanibeauty.com/makeup/face-
makeup/foundation/luminous-silk-foundation/A041.html");
    } else {
```

```
       recProducts.push("https://www.lancome-usa.com/makeup/face-
makeup/foundation/teint-idole-ultra-wear-24h-longwear-
foundation/1000554.html");

recProducts.push("https://www.esteelauder.com/product/643/22830/product-
catalog/makeup/face/foundation/double-wear/stay-in-place-makeup-spf-10");
    }

    const recommendations = {
      skinTone: "Based on your answers",
      undertone: undertone,
      coverage: coverage,
      finish: finish,
      tips: [
        `For ${skinType.toLowerCase()} skin, prep with a suitable primer.`,
        `Since you prefer ${coverage.toLowerCase()} coverage, apply in thin
layers to build it up.`,
        `A ${finish.toLowerCase()} finish is great for your skin type!`
      ],
      links: recProducts
    };

    console.log("[QUIZ] Recommendations generated for:", skinType, undertone);
    res.json({ success: true, recommendations });

  } catch (error) {
    console.error("[QUIZ ERROR]", error);
    res.status(500).json({ error: "Failed to process quiz" });
  }
});

// ========================
// CONTACT ENDPOINT
// ========================
app.post("/contact", async (req, res) => {
  try {
    const { name, email, message } = req.body;

    if (!name || !email || !message) {
      return res.status(400).json({ error: "All fields are required" });
    }

    // Save to Firebase "messages" collection
    await db.collection("messages").add({
      name,
      email,
      message,
      sentAt: new Date().toISOString()
```

```
    });

    console.log(`[CONTACT] Message received from ${email}`);
    res.json({ success: true });

  } catch (error) {
    console.error("[CONTACT ERROR]", error);
    res.status(500).json({ error: "Failed to send message" });
  }
});

// 404 logger (must be last)
app.use((req, res) => {
  console.log("[404]", req.method, req.url);
  res.status(404).send("Not Found");
});

app.listen(5000, () => console.log("☑ Server running on
http://localhost:5000"));
```

C.2 Frontend Homepage (index.html)

This file serves as the main entry point for the user. It includes the navigation, hero section, and the JavaScript for the theme toggle and the AI chatbot interface.

```
```html
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width,initial-scale=1" />
  <title>Foundation Finder • Home</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div class="wrapper">
    <nav class="nav">
      <div class="brand">Foundation Finder</div>
      <div class="nav-menu">
        <a href="index.html">Home</a>
        <a href="quiz.html">Take Quiz</a>
        <a href="upload.html">Upload Photo</a>
        <a href="about.html">About Us</a>
        <a href="contact.html">Contact</a>
        <a href="matches.html">My Matches</a>
```
```

```
      </div>
      <button id="theme-toggle" style="background:none; border:none; font-
size:20px; cursor:pointer;" title="Toggle Theme">🎨</button>
    </nav>

    <section class="center">
      <div class="card fade-up hero-glow" style="text-align:center; max-
width:760px">
        <h1 class="gradient-text">Find Your Perfect Base</h1>
        <p class="tagline">Answer 10 quick questions or upload a photo to get
a tailored foundation match.</p>
        <div style="display:flex; gap:12px; justify-content:center; flex-
wrap:wrap">
          <a class="btn accent" href="quiz.html">Take the Quiz</a>
          <a class="btn secondary" href="upload.html">Upload Photo</a>
          <a class="btn" href="about.html">Learn About Us</a>
        </div>
      </div>
    </section>

    <p class="footer">© 2025 Foundation Finder</p>
  </div>

  <div id="chat-widget" style="display: none; position: fixed; bottom: 80px;
right: 20px; width: 300px; background: white; border-radius: 15px; box-shadow:
0 5px 20px rgba(0,0,0,0.2); overflow: hidden; z-index: 1000; border: 1px solid
#e5e7eb;">
    <div style="background: var(--brand); color: white; padding: 15px; font-
weight: bold; display: flex; justify-content: space-between;">
      <span>💄 Beauty Advisor</span>
      <button onclick="toggleChat()" style="background:none; border:none;
color:white; cursor:pointer;">✕</button>
    </div>
    <div id="chat-messages" style="height: 250px; padding: 15px; overflow-y:
auto; background: #f9fafb; display: flex; flex-direction: column; gap: 10px;">
      <div style="background: #e5e7eb; padding: 8px 12px; border-radius: 15px;
align-self: flex-start; max-width: 80%;">
        Hello! Ask me anything about your foundation or makeup tips! ✨
      </div>
    </div>
    <div style="padding: 10px; border-top: 1px solid #eee; display: flex;">
      <input type="text" id="user-input" placeholder="Type your question..."
style="flex: 1; padding: 8px; border: 1px solid #ddd; border-radius: 20px;
outline: none;">
      <button onclick="sendMessage()" style="background: var(--accent);
border: none; padding: 8px 15px; margin-left: 8px; border-radius: 20px;
cursor: pointer;">➤</button>
    </div>
```

```
  </div>

  <div id="robot-icon" style="position: fixed; bottom: 75px; right: 20px;
width: 60px; height: 60px; z-index: 1001; display: none; pointer-events:
none;">
  <img src="robot-icon.jpg" alt="AI Robot" style="width: 100%; height: 100%;
object-fit: contain;">
  </div>
  <button onclick="toggleChat()" style="position: fixed; bottom: 20px; right:
20px; background: var(--brand); color: white; width: 50px; height: 50px;
border-radius: 50%; border: none; font-size: 24px; cursor: pointer; box-
shadow: 0 4px 10px rgba(0,0,0,0.2); z-index: 1000;">
    💬
  </button>

  <script src="theme.js"></script>
  <script>
    // Chat & Robot Script
    const chatWidget = document.getElementById('chat-widget');
    const chatMessages = document.getElementById('chat-messages');
    const userInput = document.getElementById('user-input');
    const robotIcon = document.getElementById('robot-icon');

    function toggleChat() {
      const isChatHidden = chatWidget.style.display === 'none';
      chatWidget.style.display = isChatHidden ? 'block' : 'none';
      robotIcon.style.display = isChatHidden ? 'block' : 'none';

      if (isChatHidden) {
        robotIcon.classList.add('robot-float');
      } else {
        robotIcon.classList.remove('robot-float');
      }
    }

    function sendMessage() {
      const msg = userInput.value.trim();
      if (!msg) return;

      appendMessage(msg, 'user');
      userInput.value = '';

      const loadingId = 'loading-' + Date.now();
      appendMessage('Thinking... 🤔', 'bot', loadingId);

      setTimeout(() => {
        document.getElementById(loadingId).remove();
```

```
        let reply = "That's a great question! As an AI, I recommend checking
our 'Take Quiz' page for personalized advice.";
        if (msg.toLowerCase().includes('hello')) reply = "Hi there! How can I
help you with your makeup today?";
        if (msg.toLowerCase().includes('foundation')) reply = "Foundation is
best applied starting from the center of your face and blending outwards!";
        appendMessage(reply, 'bot');
      }, 1000);
    }

    function appendMessage(text, sender, id = null) {
      const div = document.createElement('div');
      if (id) div.id = id;
      div.style.padding = '8px 12px';
      div.style.borderRadius = '15px';
      div.style.maxWidth = '80%';
      div.style.alignSelf = sender === 'user' ? 'flex-end' : 'flex-start';
      div.style.background = sender === 'user' ? 'var(--accent)' : '#e5e7eb';
      div.style.color = sender === 'user' ? '#000' : '#000';
      div.textContent = text;
      chatMessages.appendChild(div);
      chatMessages.scrollTop = chatMessages.scrollHeight;
    }

    // Allow "Enter" to send
    userInput.addEventListener('keypress', (e) => {
      if (e.key === 'Enter') sendMessage();
    });
  </script>
</body>
</html>
```
```
```

C.3 Frontend Authentication (auth.js)

This module uses Firebase Authentication (client-side) to manage user signup, sign-in, and logout. It also dynamically updates the navigation bar based on auth state.

```
```js
import { initializeApp } from
"https://www.gstatic.com/firebasejs/11.0.1/firebase-app.js";
import {
  getAuth,
  createUserWithEmailAndPassword,
```
```

```
  signInWithEmailAndPassword,
  signOut,
  onAuthStateChanged
} from "https://www.gstatic.com/firebasejs/11.0.1/firebase-auth.js";

// Your Config (Same as in your other files)
const firebaseConfig = {
  apiKey: "AIzaSyCLuH8sXI6JaR-seKfSO1PYF1ux6zvD1Hw",
  authDomain: "foundation-finder-3dd16.firebaseapp.com",
  projectId: "foundation-finder-3dd16",
  storageBucket: "foundation-finder-3dd16.firebasestorage.app",
  messagingSenderId: "95783410327",
  appId: "1:95783410327:web:afe1c287dd684a142ce7d2"
};

const app = initializeApp(firebaseConfig);
const auth = getAuth(app);

// --- DOM Elements ---
const signinForm = document.getElementById('signinForm');
const signupForm = document.getElementById('signupForm');
const authMsg = document.getElementById('auth-msg');
const navMenu = document.querySelector('.nav-menu');

// --- 1. Handle Sign Up ---
if (signupForm) {
  signupForm.addEventListener('submit', async (e) => {
    e.preventDefault();
    const email = document.getElementById('signup-email').value;
    const password = document.getElementById('signup-password').value;

    try {
      await createUserWithEmailAndPassword(auth, email, password);
      window.location.href = 'index.html'; // Redirect to home on success
    } catch (error) {
      showError(error.message);
    }
  });
}

// --- 2. Handle Sign In ---
if (signinForm) {
  signinForm.addEventListener('submit', async (e) => {
    e.preventDefault();
    const email = document.getElementById('signin-email').value;
    const password = document.getElementById('signin-password').value;

    try {
```

```
      await signInWithEmailAndPassword(auth, email, password);
      window.location.href = 'index.html'; // Redirect home
    } catch (error) {
      showError("Invalid email or password.");
    }
  });
}

// --- 3. Handle Global Auth State (Update Navbar) ---
onAuthStateChanged(auth, (user) => {
  if (user) {
    // If logged in, add a Logout button to the nav
    const logoutLink = document.createElement('a');
    logoutLink.href = "#";
    logoutLink.textContent = "Logout";
    logoutLink.addEventListener('click', () => signOut(auth));

    // Remove generic "Login" link if it exists
    const loginLink = document.querySelector('a[href="login.html"]');
    if (loginLink) loginLink.remove();

    navMenu.appendChild(logoutLink);
  }
});

// Helper to show errors
function showError(msg) {
  if (authMsg) {
    authMsg.textContent = msg.replace('Firebase: ', '');
  } else {
    alert(msg);
  }
}
```

C.4 Main Stylesheet (style.css)

A sample of the CSS used to define the project's visual identity, including the glass-morphism cards, theme variables, and animations.

```
```css
/* ---------- Base ---------- */
:root{
  --brand:#111827;          /* deep gray for headings */
  --accent:#eab308;         /* warm golden accent */
  --ink:#222;               /* body text */
```
```

```
  --muted:#6b7280;
  --card: rgba(255, 255, 255, 0.642);
  --glass: rgba(255, 255, 255, 0.553);
  --radius: 18px;
  --shadow: 0 10px 30px rgba(0,0,0,.18);
}

*{box-sizing:border-box}
html,body{height:100%}
body{
  margin:0;
  font-family: ui-sans-serif, system-ui, -apple-system, Segoe UI, Roboto,
Inter, "Helvetica Neue", Arial, "Noto Sans", "Apple Color Emoji","Segoe UI
Emoji";
  color: var(--ink);
  line-height:1.55;
  background: #000 url('background.png') center/cover fixed no-repeat;
  position: relative;
}

/* 💧 Removed dark overlay to keep background bright */
body::before{
  content:none;
}

/* ---------- Layout ---------- */
.wrapper{
  max-width: 1100px;
  margin: 24px auto;
  padding: 16px;
}
.card{
  background: rgba(0,0,0,0.55);   /* dark overlay only for content */
  color: #fff;
  box-shadow: var(--shadow);
  border-radius: var(--radius);
  padding: 28px;
  backdrop-filter: blur(4px);
}

/* ---------- Navbar ---------- */
.nav{
  display:flex; align-items:center; justify-content:space-between;
  gap:16px;
  background: var(--glass);
  border-radius: var(--radius);
  padding: 10px 14px;
  box-shadow: var(--shadow);
```

```
}
.brand{
  font-weight:800; letter-spacing:.3px;
  color: var(--brand);
  font-size: 20px;
}
.nav a{
  text-decoration:none; color:#111; font-weight:600;
  padding:10px 14px; border-radius:12px;
  transition:.18s ease;
}
.nav a:hover{ background: rgba(0,0,0,.06); }
.nav-menu{ display:flex; gap:6px; flex-wrap:wrap; }

/* ---------- Hero & Buttons ---------- */
.center{
  display:grid; place-items:center;
  min-height: calc(100vh - 120px);
}
.hero{
  background: rgba(0,0,0,0.55);  /* dark behind hero text */
  color: #fff;
  padding: 30px;
  border-radius: var(--radius);
}
h1,h2{ color: #fff; margin:0 0 10px }
.tagline{ color: #e5e5e5; margin-bottom:24px }

.btn{
  appearance:none; border:0; cursor:pointer;
  background: #111; color:#fff;
  padding: 12px 18px; border-radius: 14px;
  font-weight:700; letter-spacing:.2px;
  transition: transform .12s ease, box-shadow .12s ease, background .2s ease;
  box-shadow: 0 6px 16px rgba(0,0,0,.18);
}
.btn:hover{ transform: translateY(-1px); }
.btn.secondary{ background:#fff; color:#111; border:1px solid #e5e7eb }
.btn.accent{ background: var(--accent); color:#111 }

/* ---------- Quiz ---------- */
.q{
  margin: 22px 0;
}
.q-title{
  font-weight:800; margin-bottom:10px; color:#fff;
}
.options{
```

```
  display:flex; gap:10px; flex-wrap:wrap;
}
.option{
  position:relative;
}
.option input{
  position:absolute; inset:0; opacity:0; pointer-events:none;
}
.option label{
  display:inline-block; background:#fff; border:1.5px solid #e5e7eb;
  padding:10px 14px; border-radius: 14px; font-weight:700; cursor:pointer;
  transition:.18s ease;
  color:#111;
}
.option input:checked + label{
  border-color: var(--accent); box-shadow: 0 0 0 4px rgba(234,179,8,.25);
}
.helper{ color: #ddd; font-size: 13px; margin-top:4px }

.result{
  margin-top:18px; padding:18px; border-radius:16px;
  background: #fff; border:1.5px dashed #d1d5db;
  color:#111;
}

/* ---------- Forms (Upload / Contact) ---------- */
.form{
  display:grid; gap:12px;
}
.input, .textarea{
  width:100%; padding:12px 14px; border-radius:14px;
  border:1.5px solid #e5e7eb; background:#fff; font-size:16px;
}
.textarea{ min-height:140px; resize:vertical }
.form .row{ display:flex; gap:12px; flex-wrap:wrap }
.badge{ display:inline-flex; align-items:center; gap:8px; font-weight:700;
  padding:8px 12px; border-radius: 999px; background:#fff; border:1px solid
#e5e7eb }

/* ---------- Footer ---------- */
.footer{
  text-align:center; color:#0d0d0d04; font-size:14px; margin: 18px 0 8px;
}

/* ---------- Loading Spinner ---------- */
.spinner {
  width: 50px;
  height: 50px;
```

```
  border: 5px solid rgba(255, 255, 255, 0.2);
  border-left-color: var(--accent);
  border-radius: 50%;
  animation: spin 1s linear infinite;
  margin: 20px auto 15px;
}

@keyframes spin {
  to { transform: rotate(360deg); }
}

.loading-text {
  text-align: center;
  color: #666;
  font-weight: 500;
  animation: pulse 1.5s ease-in-out infinite;
}

@keyframes pulse {
  0%, 100% { opacity: 0.6; }
  50% { opacity: 1; }
}

/* ---------- Light Mode Overrides ---------- */
body.light-mode {
  background: #f9fafb; /* Solid light gray background, no image */
  color: #111827;
}

body.light-mode .nav,
body.light-mode .card,
body.light-mode .hero {
  background: rgba(255, 255, 255, 0.85);
  color: #111827;
  border: 1px solid #e5e7eb;
  box-shadow: 0 4px 12px rgba(0,0,0,0.05);
}

body.light-mode h1,
body.light-mode h2,
body.light-mode .q-title {
  color: #111827;
}

body.light-mode .tagline,
body.light-mode .helper {
  color: #6b7280;
}
```

```
body.light-mode .nav a:hover {
  background: rgba(0,0,0,0.05);
}

/* ⬇️⬇️⬇️ NEW CODE ADDED HERE ⬇️⬇️⬇️ */

/* ---------- About Page Features ---------- */
.features-grid {
  display: flex;
  gap: 20px;
  margin-top: 40px;
  flex-wrap: wrap;
}
.feature-item {
  flex: 1;
  min-width: 200px;
  background: rgba(0,0,0,0.2);
  padding: 20px;
  border-radius: 12px;
  text-align: center;
}
body.light-mode .feature-item {
  background: rgba(255,255,255,0.5);
  border: 1px solid #e5e7eb;
}
.feature-icon {
  font-size: 40px;
  margin-bottom: 10px;
  display: block;
}
/* ---------- Robot Animation ---------- */
@keyframes float {
  0% { transform: translateY(0px); }
  50% { transform: translateY(-8px); }
  100% { transform: translateY(0px); }
}

.robot-float {
  animation: float 2s ease-in-out infinite;
}
/* ---------- ✨ New Beauty Upgrades ✨ ---------- */

/* 1. Gold Gradient Text Effect */
.gradient-text {
  background: linear-gradient(to right, #fff, var(--accent), #fff);
  background-size: 200% auto;
  -webkit-background-clip: text;
```

```
  -webkit-text-fill-color: transparent;
  animation: shimmer 3s linear infinite;
}

@keyframes shimmer {
  to { background-position: 200% center; }
}

/* 2. Entrance Float Animation */
.fade-up {
  animation: fadeUp 1s ease-out forwards;
  opacity: 0;
  transform: translateY(30px);
}

@keyframes fadeUp {
  to { opacity: 1; transform: translateY(0); }
}

/* 3. Premium Card Glow */
.hero-glow {
  box-shadow: 0 15px 35px rgba(0,0,0,0.3), 0 0 40px rgba(234, 179, 8, 0.2); /*
Accent color glow */
  border: 1px solid rgba(255, 255, 255, 0.1);
}
/* ---------- ✨ Professional Auth Upgrades ✨ ---------- */
.input-group {
  position: relative;
  margin-bottom: 15px;
}
.input-group i {
  position: absolute;
  left: 15px;
  top: 50%;
  transform: translateY(-50%);
  color: var(--muted);
  font-size: 14px;
}
.input-group .input {
  padding-left: 40px; /* Make room for the icon */
  transition: all 0.3s ease;
  border: 1px solid #e5e7eb;
}
.input-group .input:focus {
  border-color: var(--accent);
  box-shadow: 0 0 0 4px rgba(234, 179, 8, 0.1); /* Subtle gold glow */
}
```

```
/* Social Login Buttons */
.social-login {
  margin-top: 25px;
  text-align: center;
}
.divider {
  display: flex;
  align-items: center;
  color: var(--muted);
  margin: 20px 0;
  font-size: 13px;
}
.divider::before, .divider::after {
  content: "";
  flex: 1;
  height: 1px;
  background: #e5e7eb;
  margin: 0 10px;
}
.btn-google {
  background: #fff;
  color: #333;
  border: 1px solid #ddd;
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 10px;
  width: 100%;
  font-weight: 600;
}
.btn-google:hover { background: #f9fafb; }

/* Extra Links */
.auth-links {
  display: flex;
  justify-content: space-between;
  font-size: 13px;
  margin-top: 15px;
  color: var(--muted);
}
.auth-links a { color: var(--muted); text-decoration: none; }
.auth-links a:hover { text-decoration: underline; }
/* ---------- Testimonials Section ---------- */
.testimonials-section {
  margin-top: 50px;
  text-align: center;
}
.testimonials-section h3 {
```

```
  color: #fff;
  margin-bottom: 30px;
  font-size: 1.5rem;
}
body.light-mode .testimonials-section h3 { color: var(--brand); }

.testimonial-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
}

.testimonial-card {
  background: rgba(255, 255, 255, 0.05);
  border: 1px solid rgba(255, 255, 255, 0.1);
  padding: 20px;
  border-radius: 16px;
  text-align: left;
  backdrop-filter: blur(5px);
  transition: transform 0.3s ease;
}
body.light-mode .testimonial-card {
  background: rgba(255, 255, 255, 0.6);
  border-color: #e5e7eb;
  box-shadow: 0 4px 15px rgba(0,0,0,0.05);
}
.testimonial-card:hover {
  transform: translateY(-5px);
  border-color: var(--accent);
}

.stars { color: #eab308; margin-bottom: 10px; font-size: 14px; }
.testimonial-text {
  font-size: 15px;
  line-height: 1.5;
  margin-bottom: 15px;
  font-style: italic;
  color: #e5e7eb;
}
body.light-mode .testimonial-text { color: var(--ink); }

.user-info {
  display: flex;
  align-items: center;
  gap: 10px;
  font-weight: bold;
  font-size: 14px;
}
```

```
.user-avatar {
  width: 30px;
  height: 30px;
  background: var(--accent);
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;
  color: #111;
  font-size: 12px;
}
```